

The Mega Study Guide

Everything for both exams — taught from zero, step by step

FCS 5DV037 Fundamentals of Computer Science (Datavetenskapens grunder)

PL 5DV086 Programming Languages (Programspråk)

BOTH = material that is examined in both courses

Every Haskell example in this guide was compiled and run with GHC 9.4 while the PDF was built, and the output you see printed under it is the real output. Every grammar, automaton, CYK table and reduction was checked by a program (see [guide/verify/](#)) — grammars against every string up to length 14. Nothing here is “from memory only”.

Contents

How to use this guide (read this first)

What the two exams actually look like

The plan for the next days

Part I The shared foundation — languages, automata, regular expressions, grammars **BOTH** exams

1. Strings, languages and the notation you must be able to read

- 1.1 Alphabets and strings
- 1.2 Languages and operations on them
- 1.3 Reading set-builder notation (the exams are full of it)
- 1.4 Proof words you will need

2. Finite automata: DFA and NFA **BOTH**

- 2.1 DFA — deterministic finite automaton
- 2.2 What a finite automaton *cannot* do
- 2.3 Running two DFAs at once: the product construction
- 2.4 NFA — nondeterministic finite automaton
- 2.5 NFA $\rightarrow$ DFA: the subset construction
- 2.6 Verification of this chapter

3. Regular expressions **BOTH**

- 3.1 Definition (course syntax)
- 3.2 Reading regular expressions
- 3.3 Writing regular expressions — recipes
- 3.4 Useful identities
- 3.5 Regular expression $\rightarrow$ NFA (Thompson's construction)
- 3.6 Automaton $\rightarrow$ regular expression (state elimination)
- 3.7 Regular expressions in real programming languages **PL**
- 3.8 Verification of this chapter

4. Minimizing a DFA **FCS**

5. Closure and decision properties of regular languages **FCS**

- 5.1 The closure properties and their one-line proofs
- 5.2 True/false facts (FCS Q1 style — -0.5 per wrong answer!)
- 5.3 Decision properties
- 5.4 The $\star$ question: infinite unions (FCS 2022 Q9)

6. The pumping lemma for regular languages **FCS**

7. Context-free grammars **BOTH**

- 7.1 Definitions
- 7.2 The grammar recipe book
- 7.3 The PL exam questions, solved slowly
- 7.4 Ambiguity

8. Precedence and associativity **PL**

8.1 Common rules

8.2 How a grammar encodes them

9. Pushdown automata **FCS**

- 9.1 CFG $\rightarrow$ PDA
- 9.2 Deterministic PDAs

10. Chomsky normal form and the CYK algorithm **BOTH**

- Part A — Chomsky normal form
- Part B — The CYK algorithm

11. Pumping lemma for CFLs, and closure **FCS**

- 11.1 Closure properties of CFLs

Part II Computability and complexity **FCS** only

12. Turing machines and the language hierarchy **FCS**

- 12.1 Turing machines computing functions — composition (FCS Q6, 6 p)
- 12.2 Variants — all equally powerful
- 12.3 The universal Turing machine
- 12.4 The hierarchy (FCS Q3, 8 p)

13. Decidability, countability, the halting problem **FCS**

- 13.1 Recursive vs recursively enumerable
- 13.2 Countability and diagonalization
- 13.3 The halting problem
- 13.4 Reductions

14. Complexity: P, NP and NP-completeness **FCS**

- 14.1 Definitions
- 14.2 SAT $\rightarrow$ 3SAT $\rightarrow$ CLIQUE, briefly
- 14.3 FCS $\star$ Q8: $\diamond$ -EQUATION is NP-complete
- 14.4 FCS $\star$ Q10: NP is closed under $\cup$ and $\cap$

Part III Haskell from zero **PL** — 20 points on every exam

15. Haskell basics: types, functions, lists **PL**

- 15.1 Using GHCi
- 15.2 Types
- 15.3 Defining functions
- 15.4 Lists: the one data structure you must master

16. Recursion and accumulators **PL**

- 16.1 Pattern 1 — plain recursion
- 16.2 Pattern 2 — accumulators and tail recursion

17. Higher-order functions: map, filter, fold, lambda **PL**

- 17.1 Lambdas, currying, sections, composition
- 17.2 map and filter — write them yourself
- 17.3 Folds
- 17.4 Pipelines — how Haskell programmers really write

18. Data types, type classes, laziness + problem bank **PL**

- 18.1 Algebraic data types

- 18.2 Recursive data types and type constructors
- 18.3 Type classes (ad hoc polymorphism)
- 18.4 An ADT in Haskell (information hiding via modules)
- 18.5 Laziness
- 18.6 Problem bank (verified answers)

Part IV Programming-language concepts PL — the “explain” questions

19. Names, binding, scope, variables and types PL

- 19.1 Paradigms in one table
- 19.2 Variables and binding
- 19.3 “Variables” in functional vs imperative languages (2023-05-05 Problem 5, 6 p)
- 19.4 Scope
- 19.5 Types and type systems

20. Control structures and subprograms PL

- 20.1 Control structures (lecture 10)
- 20.2 Subprograms
- 20.3 Parameter passing: by value, by reference and the others

21. Abstraction, ADTs and object-orientation PL

- 21.1 Definitions (these are the lecture’s own words, expanded)
- 21.2 The same stack, showing both ideas (compiles and runs)
- 21.3 The neighbours — define these precisely too
- 21.4 The formal ADT specification (lecture 11)
- 21.5 Object-oriented programming (lecture 11)

22. Interpreters: lexer, parser, executor PL

- 22.1 Lexer (scanner)
- 22.2 Parser
- 22.3 Between parser and executor: static (semantic) analysis
- 22.4 Executor (the interpreting part)
- 22.5 See it: a complete tiny interpreter (Haskell, runs)
- 22.6 Precedence and associativity (asked on both 2023 exams, 6 p)

23. Concurrency, exceptions and events PL

- 23.1 Concurrency
- 23.2 Exception handling
- 23.3 Events

24. Functional and logic languages PL

- 24.1 Functional languages
- 24.2 Logic programming (Prolog)

Part V Real exams solved, practice exams, cheat sheets

25. The FCS exam (5DV037), every question answered FCS

- 25.1 Practice exam (new questions)

26. The PL exams (5DV086), every question answered PL

- 26.1 Tentamen 2023-03-16
- 26.2 Tentamen 2023-05-05
- 26.3 Your last exam (what you told me)
- 26.4 Practice exam (60 p, 36 to pass) — 4 hours, no aids

Appendix A. Cheat sheets

Appendix B. Swedish–English glossary (PL exam is in Swedish)

How to use this guide (read this first)

You have two exams with one day in between. They overlap a lot: grammars, regular expressions and the idea of “a language as a set of strings” show up in both. So this guide is built as **one** course, not two:

1. **Part I — The shared foundation** **BOTH**. Strings and languages, automata, regular expressions, context-free grammars, precedence & associativity, parsing. You need this for both exams, so it comes first.
2. **Part II — Computability & complexity** **FCS**. Turing machines, decidability, the halting problem, P and NP.
3. **Part III — Haskell from zero** **PL**. Written as if you have never seen it. Ends with every kind of function the exam has asked you to write.
4. **Part IV — Programming-language concepts** **PL**. Abstraction, encapsulation, polymorphism, scope, binding, parameter passing, interpreters, concurrency, exceptions, logic programming — with *model answers* written the way an examiner wants to read them.
5. **Part V — Real exams, solved**, and new practice exams with solutions.
6. **Appendix** — one-page cheat sheets and a Swedish-English glossary (the PL exam is in Swedish).

HOW EACH CHAPTER WORKS

Every chapter goes: **intuition first** (what is this, in plain words) → **the formal definition** (what you write on the exam) → **worked examples** (slowly, every step shown) → **recipes** (a checklist you can follow under stress) → **traps** (where students lose points) → **practice** with solutions.

The coloured boxes mean: **Intuition**, **Definition**, **Theorem**, **Recipe**, **Exam tip / Model answer**, **Trap**.

What the two exams actually look like

PL Programspråk 5DV086 (H. Björklund)

Both past exams you have (2023-03-16 and 2023-05-05) have **7 problems, 60 points, 36 needed to pass** (60%). They are almost the same exam twice:

#	2023-03-16	2023-05-05	Pts	Where in this guide
1	Which strings does grammar G generate?	Build a CFG for $\{a^n b^m \mid n \neq 2m\}$	10	Ch. 7 (CFGs)
2	Build a CFG for “equally many a and b, every prefix has at least as many a as b” — identical both times		10	Ch. 7
3	Explain <i>precedence</i> and <i>associativity</i> — identical		6	Ch. 8
4	Parts of an interpreter (scanner, parser, executor) and their data structures — identical		8	Ch. 22
5	Pass-by-value vs pass-by-reference	“Variables” in functional vs imperative languages	6	Ch. 19, 20
6	Write <code>compress</code> in Haskell	Explain/evaluate code with <code>filter</code> , <code>foldl</code>	10	Ch. 15–18

7	Write <code>myRev</code> (reverse a list) in $O(n)$, no library functions — identical	10	Ch. 16
---	---	----	--------

So the exam is roughly **20 points grammars + 20 points Haskell + 20 points concepts**. If you can do Problem 2, Problem 3, Problem 4 and Problem 7 perfectly you already have 34 points — almost a pass — and those four were *word-for-word the same* on both exams. An older exam (2017) also asked for a **tail-recursive Fibonacci** function (Ch. 16).

THE COURSE CHANGED IN 2024

The official syllabus for 5DV086 was revised from 2024-01-01. The listed literature is now *Crafting Interpreters* (Robert Nystrom) and *Learn You a Haskell for Great Good!* (Lipovača). The content description is unchanged: variables, binding, scope, types, type checking, control abstraction, subprograms, parameter passing, abstract data types, the paradigms (imperative, OO, functional, logic), concurrency and exceptions, with special emphasis on functional programming (recursive functions, recursive data types, type constructors). Because of the new book, expect more weight on **how interpreters work** (scanning, parsing, trees, environments) — Chapter 22 covers it and includes a small working interpreter in Haskell. When you find your own recent exams, compare them against the table above.

FCS Datavetenskapens grunder 5DV037 (M. Berglund)

The past exam (2022 content, 2025 cover sheet) has **10 questions, 50 points, 4 hours**, and you may bring only pens and an English dictionary. **A page of definitions is attached:** the two pumping lemmas, the NPDA definition and the Turing-machine definition. You do not need to memorise those statements — you need to know how to *use* them.

- Grade 3: 50% (25/50). Grade 4: 67% *and* ≥ 1 of the 3 ☆-points. Grade 5: 83% *and* ≥ 2 ☆-points.
- The three ☆-questions are worth only 1 point each. **Do them last.**

#	Topic	Pts	Chapter
1	True/false (−0.5 per wrong answer!) on closure, P/NP, decidability	6	5, 11, 13, 14
2	Argue regular languages are closed under complement, $\cap$, concatenation, $\cup$	5	5
3	Hierarchy of 5 language classes: name, machine, example language	8	12
4	Prove a language is not context-free (pumping lemma)	8	11
5	Is the grammar in CNF? Run CYK on <code>abcbab</code>	8	10
6	Build a TM that computes $g(f(v))$ from TMs for f and g	6	12
7	Why nondeterministic TMs are no more powerful than deterministic ones	6	12
8☆	Prove a ◇-EQUATION problem is NP-complete	1	14
9☆	Is an infinite union of regular languages regular?	1	5
10☆	Define NP; prove NP is closed under $\cup$ and $\cap$	1	14

Questions 1–7 (47 points) are all “standard technique” questions. Every one of them has a recipe in this guide.

The plan for the next days

Because the PL exam relies heavily on grammars, and the FCS exam relies on grammars and automata, do Part I first no matter which exam comes first.

Block	Do this	Why
1	Ch. 1 (notation) → Ch. 7 (CFGs) → Ch. 8 (precedence/associativity)	30 PL points and ~16 FCS points depend on grammars
2	Ch. 15–17 (Haskell) — type every example into GHCi if you can	20 PL points; needs practice, not reading
3	Ch. 2–6 (automata, regex, closure, pumping lemma)	FCS Q1, Q2, Q3, Q9
4	Ch. 9–11 (PDA, CNF, CYK, CF pumping)	FCS Q4, Q5 = 16 points
5	Part IV (PL concepts) — learn the model answers	20 PL points, pure memorisation of understanding
6	Ch. 12–14 (TM, decidability, NP)	FCS Q3, Q6, Q7 and ☆
7	Part V: do the real exams under time pressure, then the practice exams	Exam technique
8 (last)	The separate Priority Pack PDF: re-read what you lost points on, then do its Grill on paper	Final refresh the day before

WHY YOU LOST POINTS BEFORE — AND HOW TO STOP

- **Blank answers get zero.** Every question in both exams gives partial credit. Write the definition, the recipe, the first steps. “Motivate” means: say *why*, in one sentence per step.
- **Grammar questions:** test your grammar on 3–4 short strings (including the empty string!) before moving on. Chapter 7 shows exactly how.
- **Haskell questions:** always write the type signature first; always write the base case first. Then trace your function on a tiny input on paper.
- **Theory questions:** define the term, give a tiny example, give one advantage and one disadvantage. That structure alone earns most of the points.

Part I

The shared foundation — languages, automata, regular expressions, grammars
BOTH exams

1. Strings, languages and the notation you must be able to read

THE ONE IDEA BEHIND BOTH COURSES

A **language** is nothing mysterious: it is just a **set of strings**. “All strings of a’s and b’s with an even number of b’s” is a language. “All syntactically correct Java programs” is a language. The whole of formal language theory asks two questions: *how do I describe such a set with a finite recipe* (a grammar, a regular expression, a machine), and *how powerful does the recipe have to be?*

1.1 Alphabets and strings

DEFINITION

- An **alphabet** Σ is a finite, non-empty set of symbols, e.g. $\Sigma = \{a, b\}$.
- A **string** (word) over Σ is a finite sequence of symbols: $abba$.
- The **empty string** has no symbols. Linz (the FCS book) writes it λ ; the PL exam and Sipser write it ϵ . **They are the same thing.**
- $|w|$ is the length of w : $|abba| = 4$, $|\lambda| = 0$.
- $|w|_a$ is the number of a’s in w : $|abba|_a = 2$, $|abba|_b = 2$.
- w^R is w reversed: $(abc)^R = cba$.
- a^n means n copies of a : $a^3 = aaa$, $a^0 = \lambda$.

Concatenation just glues strings: if $u = ab$ and $v = ba$ then $uv = abba$. Note that $\lambda w = w\lambda = w$ — the empty string is the “do nothing” of concatenation, like 0 for addition.

Prefix, suffix, substring. u is a *prefix* of w if $w = uv$ for some v . The PL exam explains it for you: the prefixes of $abca$ are $\epsilon, a, ab, abc, abca$. A *suffix* is the same from the other end; a *substring* is any contiguous piece.

1.2 Languages and operations on them

DEFINITION

- Σ^* = the set of **all** strings over Σ , including λ . For $\Sigma = \{a, b\}$: $\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, \dots\}$.
- $\Sigma^+ = \Sigma^* \setminus \{\lambda\}$ (all non-empty strings).
- A **language** over Σ is any subset $L \subseteq \Sigma^*$.

Because languages are sets, all set operations work: union $L_1 \cup L_2$, intersection $L_1 \cap L_2$, difference $L_1 \setminus L_2$, complement $\bar{L} = \Sigma^* \setminus L$. Two extra operations only make sense for languages:

- **Concatenation:** $L_1 L_2 = \{uv \mid u \in L_1, v \in L_2\}$. Example: $\{a, ab\}\{b\} = \{ab, abb\}$.
- **Star (Kleene closure):** $L^* = \{\lambda\} \cup L \cup LL \cup LLL \cup \dots$ — glue together any number (including zero) of strings from L . Example: $\{ab\}^* = \{\lambda, ab, abab, ababab, \dots\}$.

TRAP: THREE DIFFERENT “EMPTY” THINGS

$\emptyset$ is the empty *language* (no strings at all). $\{\lambda\}$ is a language containing exactly one string, the empty one. λ itself is a *string*. So $|\emptyset| = 0$ but $|\{\lambda\}| = 1$. Also $\emptyset^* = \{\lambda\}$ (zero pieces glued together is the empty string).

1.3 Reading set-builder notation (the exams are full of it)

$\{w \in \{a, b\}^* \mid \text{condition}\}$ reads “all strings w made of a’s and b’s such that the condition holds”.

Practise translating until it is automatic:

Notation	In words	Some members	Some non-members
$\{a^n b^n \mid n \geq 0\}$	some a’s followed by <i>the same number</i> of b’s	$\lambda, ab, aabb$	$aab, ba, abab$
$\{a^n b^m \mid n \neq 2m\}$	a’s then b’s, and #a is <i>not</i> twice #b	$a, b, ab, aaab$	$\lambda, aab, aaaabb$
$\{w \mid w _a = w _b\}$	equally many a’s and b’s, <i>any order</i>	$\lambda, ab, ba, abba$	a, aab
$\{ww^R \mid w \in \{a, b\}^*\}$	even-length palindromes	$\lambda, aa, abba$	aba, ab
$\{a^{n^2} \mid n \geq 0\}$	a’s, where the count is a perfect square	λ, a, a^4, a^9	aa, aaa
$\{w \mid w _a > 2 w _b\}$	more than twice as many a’s as b’s	$a, aaab, aaba$	λ, aab

Notice the difference between $a^n b^n$ (order is forced: all a’s first) and $|w|_a = |w|_b$ (order is free). Mixing those two up is a classic way to build the wrong grammar.

EXAM HABIT

Before you build anything for a language, **write down 3 members and 3 non-members**, including the shortest ones (is λ in it?). This takes 30 seconds and catches most misunderstandings.

1.4 Proof words you will need

- **“Closed under X”**: doing X to languages in the class always gives a language in the class. “Regular languages are closed under union” = the union of two regular languages is always regular.
- **Proof by contradiction**: assume the opposite, derive something impossible. All pumping-lemma proofs and the halting-problem proof work like this.
- **Proof by construction**: show something exists by building it (e.g. build a DFA for the union).
- **Countable**: can be listed as item 1, item 2, item 3, ... (Chapter 13). Σ^* is countable: list strings by length, then alphabetically.
- $\mathbb{N} = \{0, 1, 2, \dots\}$ in this course (the pumping lemma appendix says “for all $i \in \mathbb{N}$ (including 0)”).

CHECK YOURSELF

1. Is $\lambda \in \{a^n b^m \mid n \neq 2m\}$?
2. List all strings of $\{a, b\}^*$ of length 2.
3. What is $\{a, b\}\{\lambda, a\}$?
4. What is $|aabab|_b$?

ANSWERS

1. No: $\lambda = a^0 b^0$ and $0 = 2 \cdot 0$. 2. aa, ab, ba, bb . 3. $\{a, aa, b, ba\}$. 4. 2.

2. Finite automata: DFA and NFA BOTH

INTUITION: A MACHINE WITH A TINY, FIXED MEMORY

Imagine reading a string one letter at a time, left to right, **never going back**, and you are only allowed to remember one of a *fixed, finite* number of things — say, one of 3 sticky notes. At the end you must say “yes” or “no”. That is a finite automaton. Each **state** is one sticky note: “*what do I need to remember about what I have read so far?*”

A lexer (scanner) in an interpreter is exactly this kind of machine: it reads characters and decides where a number or identifier ends. That is why this chapter matters for the PL exam too.

2.1 DFA — deterministic finite automaton

DEFINITION (LINZ)

A DFA is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where

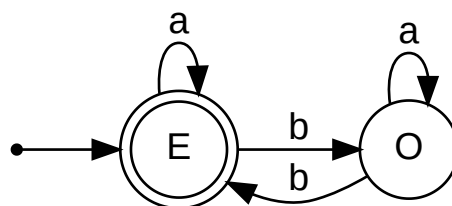
- Q is a finite set of **states**,
- Σ is the input **alphabet**,
- $\delta : Q \times \Sigma \rightarrow Q$ is the **transition function** — for *every* state and *every* symbol there is **exactly one** next state,
- $q_0 \in Q$ is the **initial state**,
- $F \subseteq Q$ is the set of **final (accepting) states**.

The extended function $\delta^*(q, w)$ is the state you end in after reading all of w from q . The **language accepted** is $L(M) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}$.

Deterministic means “no choices”: in each situation there is exactly one move. If some transitions are “missing” in a drawing, the convention is that they go to a **trap state** (a non-final state that loops to itself on everything) — you are dead and can never accept.

Example 1 — even number of b's

What must I remember? Only whether I have seen an even or odd number of b's so far. Two sticky notes: E (even) and O (odd). Reading an a changes nothing; reading a b flips.



DFA for $\{w \in \{a, b\}^* \mid |w|_b \text{ is even}\}$. Double circle = final.

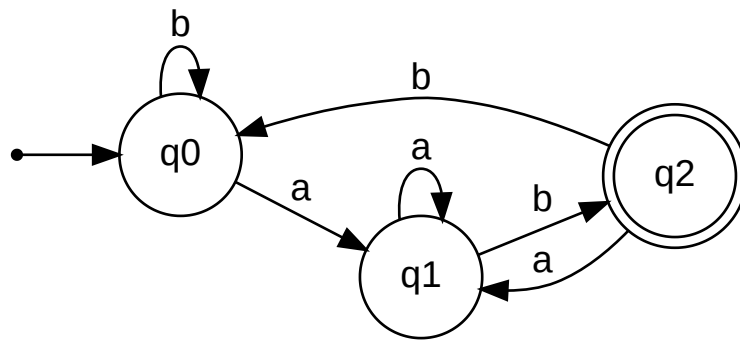
Trace $abba$: $E \xrightarrow{a} E \xrightarrow{b} O \xrightarrow{b} E \xrightarrow{a} E$. Ends in E (final) → accepted. Trace ab : $E \rightarrow E \rightarrow O \rightarrow$ rejected. ✓

Example 2 — strings that end with ab

What must I remember? How much of the pattern “ ab ” I have just seen at the end:

- q_0 : the string so far does not end with a or ab (“nothing useful”),

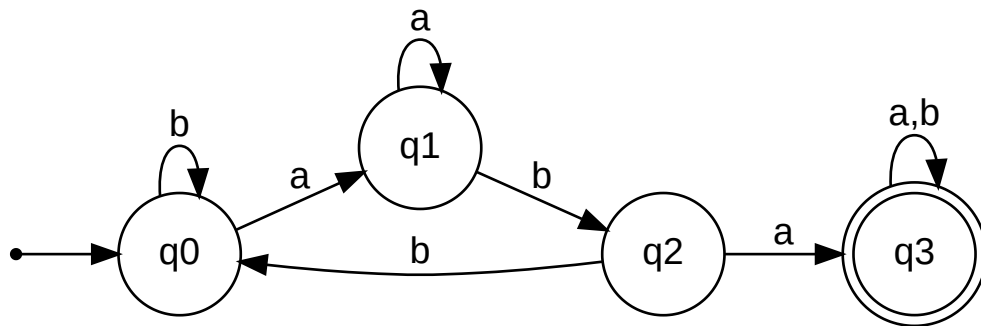
- q_1 : it ends with a (half-way),
- q_2 : it ends with ab (success — final).



DFA for strings ending in ab

The tricky arrows are the “fall-back” ones. From q_2 (ending with ab) reading a : the string now ends with $a \rightarrow q_1$, not q_0 ! From q_1 reading a : still ends with $a \rightarrow$ stay.

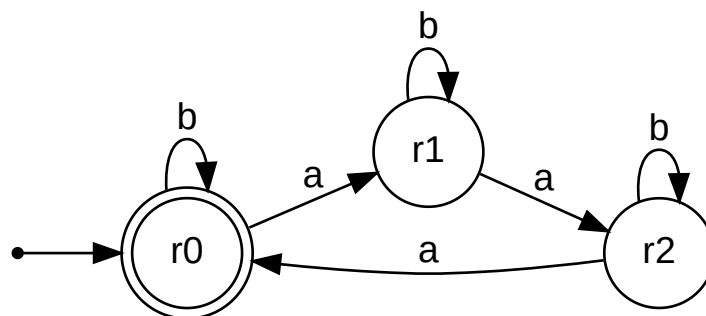
Example 3 — strings that contain aba



DFA for strings containing aba (q_3 is final and absorbing)

Once you have seen aba you can never “un-see” it, so q_3 loops on everything. From q_2 (just read ab) a b gives abb , which ends with nothing useful $\rightarrow q_0$.

Example 4 — number of a ’s divisible by 3



Counting modulo 3: remember the remainder

RECIPE: DESIGNING A DFA

1. Ask: **what is the minimum I must remember** about the prefix read so far to decide the answer later? (a remainder, the last few letters, a flag “already seen X”, a parity...)
2. Make one state per possible “memory”. Mark as final those memories that mean “accept if the string ended now”.
3. For **every** state and **every** letter, ask “what do I remember after this letter?” and draw the arrow. Count: states $\times$ letters arrows.
4. Test on the empty string and 3–4 short strings.

WHERE THE DFA POINTS GET LOST

- A missing transition in a DFA (unless you say “missing arrows go to a trap state”).
- Forgetting to decide if the **start state is final** — it is exactly when $\lambda \in L$.
- The wrong fall-back in pattern-matching DFAs (Example 2 from q_2 on a).

2.2 What a finite automaton *cannot* do

A DFA with k states can remember only k different things. So it cannot count without bound: $\{a^n b^n\}$ would require remembering n , which can be any number. That is the intuition behind the pumping lemma (Chapter 6): with more letters than states, some state must repeat (pigeonhole principle).

2.3 Running two DFAs at once: the product construction

INTUITION

To check “even number of b’s **and** ends with ab ”, run both machines side by side on the same input, with one finger on each. A state of the combined machine is a **pair** (state of M_1 , state of M_2).

PRODUCT CONSTRUCTION

Given $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$, build $M = (Q_1 \times Q_2, \Sigma, \delta, (q_1, q_2), F)$ with

$$\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a)).$$

For **intersection** take $F = F_1 \times F_2$ (both accept). For **union** take $F = \{(p, q) \mid p \in F_1 \text{ or } q \in F_2\}$.

For our example the product has $2 \times 3 = 6$ states and the only final state is (E, q_2) . This construction is the proof that regular languages are closed under $\cap$ and $\cup$ (Chapter 5).

2.4 NFA — nondeterministic finite automaton

INTUITION: A MACHINE THAT CAN GUESS

An NFA may have **several** arrows for the same letter, **none**, or arrows labelled λ that it may take without reading anything. It **accepts** if **at least one** way of choosing leads to a final state. Think of it as a machine that clones itself at every choice; if any clone ends in a final state, the answer is yes.

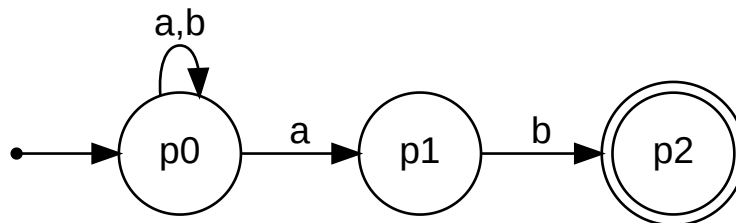
DEFINITION (LINZ)

An NFA is $M = (Q, \Sigma, \delta, q_0, F)$ like a DFA except

$$\delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow 2^Q,$$

i.e. each (state, symbol-or- λ) gives a **set** of possible next states (possibly empty). $w \in L(M)$ iff some path labelled w goes from q_0 to a state in F .

NFAs are often much easier to design. “Ends with ab ”: stay in p_0 as long as you like, and *guess* when the final ab starts:



NFA for strings ending in ab — it guesses where the last ab begins

2.5 NFA → DFA: the subset construction

THEOREM

For every NFA there is a DFA accepting the same language. So NFAs are **not more powerful** than DFAs — only more convenient. (The same question for Turing machines is FCS exam Q7, Chapter 12.)

INTUITION

Instead of guessing, keep track of **the set of all states the NFA could be in right now**. There are only finitely many such sets (at most $2^{|Q|}$), so this is a DFA whose states are sets.

RECIPE: SUBSET CONSTRUCTION

1. **λ -closure** $E(S)$: all states reachable from S using only λ -arrows (including S itself).
2. Start state of the DFA: $E(\{q_0\})$.
3. For each DFA state S found so far and each letter a : collect every state reachable from some $q \in S$ by one a -arrow, then take the λ -closure. That set is the new DFA state.
4. Repeat until no new sets appear. The empty set $\emptyset$ is a trap state.
5. A DFA state is **final** if it **contains at least one** NFA final state.

Worked example 1 (the NFA above). Output of the verifier (the table is the DFA):

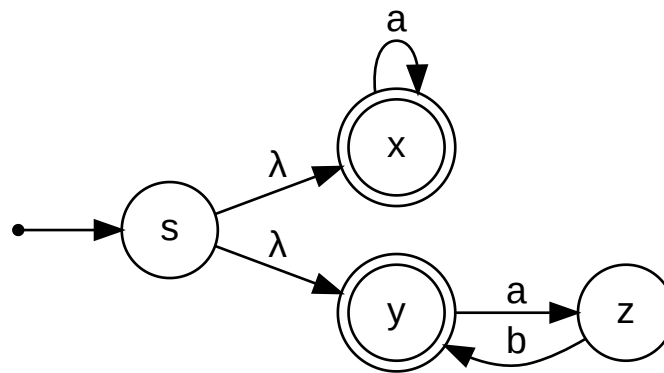
```

$ python3 verify/ch02_subset1.py # subset construction
DFA state      a          b          final?
->{p0}          {p0,p1}      {p0}
{p0,p1}        {p0,p1}      {p0,p2}
{p0,p2}        {p0,p1}      {p0}          yes
  
```

Read it row by row: from $\{p_0\}$, on a the NFA can go to p_0 or $p_1 \rightarrow \{p_0, p_1\}$; on b only to p_0 . From $\{p_0, p_1\}$ on b : $p_0 \rightarrow p_0, p_1 \rightarrow p_2 \rightarrow \{p_0, p_2\}$, which is final because it contains p_2 . Compare with the DFA in

Example 2: it is the same machine with $q_0 = \{p_0\}$, $q_1 = \{p_0, p_1\}$, $q_2 = \{p_0, p_2\}$.

Worked example 2 (with λ -moves). NFA for $a^* \cup (ab)^*$:



λ -NFA for $a^* \cup (ab)^*$

```
$ python3 verify/ch02_subset2.py
```

DFA state	a	b	final?
$\rightarrow\{s, x, y\}$	$\{x, z\}$	$\emptyset$	yes
$\{x, z\}$	$\{x\}$	$\{y\}$	yes
$\emptyset$	$\emptyset$	$\emptyset$	
$\{x\}$	$\{x\}$	$\emptyset$	yes
$\{y\}$	$\{z\}$	$\emptyset$	yes
$\{z\}$	$\emptyset$	$\{y\}$	

The start state is $E(\{s\}) = \{s, x, y\}$ — don't forget the closure! It is final (contains x), which is right because λ is in the language. On a : $x \rightarrow x, y \rightarrow z$, so $\{x, z\}$. On b : nothing $\rightarrow \emptyset$ (trap).

SUBSET-CONSTRUCTION TRAPS

- Forgetting the λ -closure of the **start** state, or after each step.
- Forgetting that $\emptyset$ is a real (trap) state of the DFA — include it, with self-loops.
- Marking only sets that are “exactly” final; the rule is **contains** a final state.

2.6 Verification of this chapter

Every machine drawn above was simulated on all strings up to length 12 and compared with its English description:

```
$ python3 verify/ch02_automata.py
```

```

[OK] even number of b's: all strings up to length 12
[OK] ends with ab: all strings up to length 12
[OK] contains aba: all strings up to length 12
[OK] #a divisible by 3: all strings up to length 12
[OK] exam Q1: starts a / ends c / odd b (product DFA with 13 states): all strings up to
length 8
[OK] product: even b AND ends with ab: all strings up to length 12
[OK] NFA for ends-with-ab
[OK] λ-NFA for a* u (ab)*
[OK] practice 1: every a followed by b: all strings up to length 12
[OK] practice 3: second-to-last letter is a (NFA)
    its DFA has 4 states: ['{p0,p1,p2}', '{p0,p1}', '{p0,p2}', '{p0}']
  
```

PRACTICE

1. DFA over $\{a, b\}$ for strings where every a is immediately followed by a b .
2. DFA for strings of odd length.
3. NFA for strings whose second-to-last letter is a ; convert it to a DFA.

SOLUTIONS (SKETCH)

1. States: OK (final, start), NEED-B (just read an a), DEAD. OK $\xrightarrow{a}$ NEED-B, OK $\xrightarrow{b}$ OK, NEED-B $\xrightarrow{b}$ OK, NEED-B $\xrightarrow{a}$ DEAD, DEAD loops.
2. Two states EVEN (start, non-final) and ODD (final), every letter swaps.
3. NFA: p_0 loops on a, b ; $p_0 \xrightarrow{a} p_1$; $p_1 \xrightarrow{a, b} p_2$ (final). The DFA has 4 states: the sets $\{p_0\}, \{p_0, p_1\}, \{p_0, p_1, p_2\}, \{p_0, p_2\}$, final = those containing p_2 (they remember the last two letters).

3. Regular expressions BOTH

INTUITION

A regular expression is a **formula that describes a language**, built from single letters using three operations: **or** (union), **then** (concatenation) and **repeat** (star). A DFA is a machine that *checks* strings; a regular expression is a pattern that *describes* them. They have exactly the same power (Kleene's theorem).

3.1 Definition (course syntax)

DEFINITION (LINZ)

1. $\emptyset$, λ and every $a \in \Sigma$ are regular expressions (primitive). They denote $\emptyset$, $\{\lambda\}$ and $\{a\}$.
2. If r_1, r_2 are regular expressions, so are $r_1 + r_2$ (union), $r_1 \cdot r_2$ or just $r_1 r_2$ (concatenation), r_1^* (star) and (r_1) .
3. Nothing else is a regular expression.

$$L(r_1 + r_2) = L(r_1) \cup L(r_2), L(r_1 r_2) = L(r_1)L(r_2), L(r^*) = (L(r))^*.$$

+ MEANS DIFFERENT THINGS IN DIFFERENT PLACES

In the course (Linz), $a + b$ means “ a **or** b ”. In programming regexes (Python, Java, grep) the “or” is written $a|b$ and a^+ means “one or more a 's” (aa^*). Sipser writes $\cup$ for union. On the FCS exam use the course notation; on the PL exam either is fine if you say which.

Precedence: star, then concatenation, then union

Exactly like arithmetic where $^$ binds tighter than $\times$, which binds tighter than $+$:

Regex	means	not
ab^*	$a(b^*)$: $a, ab, abb, \dots$	$(ab)^*$
$a + bc$	$a + (bc)$: $\{a, bc\}$	$(a + b)c$
$a + b^*$	$a + (b^*)$	$(a + b)^*$

This is a first taste of *precedence*, the topic of PL exam Problem 3 (Chapter 8).

3.2 Reading regular expressions

Read from the outside in, and list the shortest members first.

Regex	Language in words
$(a + b)^*$	all strings over $\{a, b\}$ (“anything”)
$(a + b)^* ab$	ends with ab
$(a + b)^* aba(a + b)^*$	contains aba somewhere (“anything, aba , anything”)
$a^* ba^*$	exactly one b
$a^*(\lambda + b)a^*$	at most one b

$((a + b)(a + b))^*$	even length (blocks of two letters)
$(ab + ba)^*$	cut into blocks of two, each block is ab or ba
$a^*(ba^*ba^*)^*$	an even number of b 's (b 's come in pairs, a 's anywhere)
$(b + ab)^*(\lambda + a)$	no two consecutive a 's (every a except maybe the last is followed by b)

3.3 Writing regular expressions — recipes

RECIPE: BUILDING A REGEX

- **“contains X”** $\rightarrow (a + b)^* X (a + b)^*$.
- **“starts with X” / “ends with X”** $\rightarrow X(a + b)^* / (a + b)^* X$.
- **“exactly / at most k of letter b”** $\rightarrow$ put the b 's explicitly with a^* (all the other letters, starred) between them: exactly two b 's = $a^*ba^*ba^*$.
- **Counting modulo k** (“even number of b 's”, “#a divisible by 3”) $\rightarrow$ a block that contains exactly k of the counted letter, starred, with the other letters starred in between: $b^*(ab^*ab^*ab^*)^*$.
- **“A or B or C”** $\rightarrow$ build each separately, join with $+$.
- **Forbidden pattern** (“no aa ”) $\rightarrow$ describe what every letter must be followed by, and fix the end.

The FCS 2022 exam, Q1 statement 1: “strings over $\{a, b, c\}$ that start with a , or end with c , or contain an odd number of b 's — is this regular?” **True.** Each part is regular and regular languages are closed under union. You can even write the regex:

$$a(a + b + c)^* + (a + b + c)^*c + (a + c)^*b(a + c)^*(b(a + c)^*b(a + c)^*)^*$$

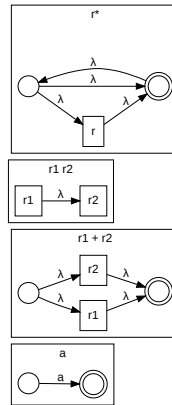
3.4 Useful identities

Identity	Why
$r + r = r$	union with itself
$\lambda r = r\lambda = r$	λ is the unit of concatenation
$\emptyset r = r\emptyset = \emptyset$	nothing to glue with $\rightarrow$ nothing (a classic trap!)
$\emptyset^* = \lambda, \lambda^* = \lambda$	zero pieces = the empty string
$(r^*)^* = r^*, r^*r^* = r^*$	repeating a repeat adds nothing
$\lambda + rr^* = r^*$	“zero times, or at least once”
$(a + b)^* = (a^*b^*)^*$	both are “everything”
$(ab)^*a = a(ba)^*$	both are $a, aba, ababa, \dots$
$(a + b)^* \neq a^* + b^*!$	ab is in the left one only

3.5 Regular expression → NFA (Thompson's construction)

INTUITION

Build a small NFA for each piece, then glue with λ -arrows, following the structure of the expression — exactly like evaluating an arithmetic expression bottom-up. Each little NFA has one start and one final state.



Building blocks: letter a , union $r1+r2$, concatenation $r1r2$, star r^*

In words: **union** — new start with λ -arrows into both machines, both finals λ -arrow to a new final.

Concatenation — λ -arrow from the final of r_1 to the start of r_2 . **Star** — new start that can skip straight to the end (zero repetitions) and a λ -arrow back to repeat.

Example: $(a + b)^*ab$. Build $a + b$ (union of two letter-machines), star it, then concatenate with a and then b . If you then remove the unnecessary λ 's you get exactly the 3-state NFA of Chapter 2 (p_0 loops on a, b , then a , then b).

3.6 Automaton → regular expression (state elimination)

RECIPE: STATE ELIMINATION (THE IDEA)

1. Add a new start state with a λ -arrow to the old start, and a new single final state with λ -arrows from all old finals.
2. Remove the other states one at a time. When you remove state q that has a self-loop s , an incoming arrow $p \xrightarrow{r_1} q$ and an outgoing arrow $q \xrightarrow{r_2} t$, add the arrow $p \xrightarrow{r_1 s^* r_2} t$ (union it with an existing $p \rightarrow t$ arrow using $+$).
3. When only start and final remain, the label on the arrow is the regex.

You do not need to be fast at this for the exam; you need to know **that** it works, because it proves:

THEOREM (KLEENE)

A language is **regular** $\Leftrightarrow$ it is accepted by some DFA $\Leftrightarrow$ by some NFA $\Leftrightarrow$ it is described by some regular expression $\Leftrightarrow$ it is generated by a **right-linear (regular) grammar** (rules of the form $A \rightarrow aB$ or $A \rightarrow a$ or $A \rightarrow \lambda$).

3.7 Regular expressions in real programming languages PL

Practical regex libraries add shorthand, but most of it is just abbreviation of the three basic operations:

Practical syntax	Meaning	Course notation
<code>a b</code>	a or b	$a + b$
<code>a*</code>	zero or more	a^*
<code>a+</code>	one or more	aa^*
<code>a?</code>	optional	$\lambda + a$
<code>[a-z]</code>	one character in the range	$a + b + \dots + z$
<code>.</code>	any single character	union of the whole alphabet
<code>a{2,3}</code>	2 or 3 a's	$aa(\lambda + a)$

A **scanner/lexer** describes each kind of token by a regex, e.g. identifier `[a-zA-Z_][a-zA-Z_0-9]*`, integer `[0-9]+`, and runs the corresponding finite automaton (Chapter 22). **Caution:** some library features, notably *back-references* like `(a*)b\1` (“the same group again”), go beyond regular languages — $\{a^nb^n\}$ is not regular, yet `(a*)b\1` matches exactly it. So “regex” in a library is not always “regular” in theory.

3.8 Verification of this chapter

```
$ python3 verify/ch03_regex.py
[OK] (a+b)*aba(a+b)*          contains aba
[OK] (a+b)*ab                  ends with ab
[OK] a*(ba*ba*)*              even number of b's
[OK] a*(λ+b)a*                 at most one b
[OK] a*ba*                     exactly one b
[OK] (b+ab)*(λ+a)              no two consecutive a's
[OK] ((a+b)(a+b))*             even length
[OK] ((a+b)(a+b)(a+b))*        length divisible by 3
[OK] b*(ab*ab*ab*)*            number of a's divisible by 3
[OK] a(a+b+c)*+(a+b+c)*c+(a+c)*b(a+c)*(b(a+c)*b(a+c)*)* exam 2022 Q1: starts a / ends c / odd #b
[OK] (a+b)*a(a+b)              second-to-last is a
[OK] (ab+ba)*                  (ab+ba)* read as: pairs, each pair ab or ba

Identities (checked as languages, strings up to length 10):
(a*)*      = a*      ✓
λ+aa*      = a*      ✓
(a+b)*     = (a*b*)*  ✓
(a*b*)*    = (a+b)*  ✓
a*a*       = a*      ✓
(ab)*a     = a(ba)*  ✓
(a+b)*     ≠ a++b*   ✗ (trap: claimed identity is FALSE, e.g. ab)
```

PRACTICE

1. Regex for strings over $\{a, b\}$ with exactly two a 's.
2. Regex for strings that do not end in b .
3. Describe $L((a + \lambda)(b + ab)^*)$ — hint: compare with “no aa ”.
4. Is $(a + b)^*a(a + b)^*$ the same as $(a + b)^*$?

SOLUTIONS

1. $b^*ab^*ab^*$. 2. $\lambda + (a + b)^*a$ (don't forget λ !). 3. strings with no aa that do *not* end in a lone a unless the whole string is a ... concretely: optional leading a , then blocks b or ab — every a after the first position is preceded by b and followed by b . 4. No: the first requires at least one a , so λ and bbb are not in it.

4. Minimizing a DFA FCS

INTUITION

Two states are **the same in practice** if, from either of them, exactly the same future strings lead to acceptance. Then you can merge them. The minimal DFA is the one where every state “means” something different.

DEFINITION

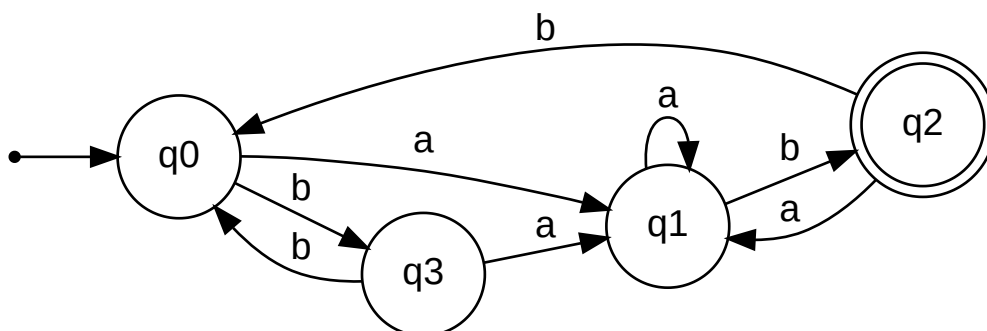
States p and q are **indistinguishable** if for every string w : $\delta^*(p, w) \in F \iff \delta^*(q, w) \in F$. Otherwise they are **distinguishable** — some string w takes one to a final state and the other to a non-final state.

RECIPE: THE MARKING (TABLE-FILLING) ALGORITHM

1. Remove states that can't be reached from the start state.
2. Draw a table with one cell for every pair of states.
3. **Round 0**: mark every pair where one state is final and the other is not (the empty string distinguishes them).
4. **Repeat**: for every unmarked pair (p, q) and every letter a : if $(\delta(p, a), \delta(q, a))$ is already marked, mark (p, q) . Stop when a whole round marks nothing new.
5. Unmarked pairs are indistinguishable: merge them. Each group becomes one state of the minimal DFA.

Worked example

This DFA accepts “ends with ab ” but has one state too many (q_3 behaves exactly like q_0):



\$ marking algorithm

[OK] 4-state DFA accepts 'ends with ab ': all strings up to length 10
round 0: mark every (final, non-final) pair: (q_0, q_2) , (q_1, q_2) , (q_2, q_3)
round 1: mark (q_0, q_1) because on 'b' they go to (q_3, q_2) which is marked
round 1: mark (q_1, q_3) because on 'b' they go to (q_2, q_0) which is marked
Unmarked pairs are indistinguishable $\rightarrow$ classes: $\{q_0, q_3\}$, $\{q_1\}$, $\{q_2\}$

Round 0 marks every pair with q_2 (the only final state). Round 1: (q_0, q_1) on b goes to (q_3, q_2) , which is marked, so mark it. Same for (q_1, q_3) . The pair (q_0, q_3) : on a both go to q_1 ; on b both go to q_0 — never reaches a marked pair. So q_0 and q_3 merge, giving the 3-state DFA of Chapter 2.

FACTS TO KNOW

- The minimal DFA of a regular language is **unique** (up to renaming states).
- Two DFAs accept the same language $\Leftrightarrow$ their minimal DFAs are the same. So equivalence of regular languages is **decidable**.

TRAPS

- Forgetting to delete unreachable states first.
- Stopping after one round — keep going until a full round marks nothing.
- Forgetting the trap state when the DFA is drawn with missing arrows.

5. Closure and decision properties of regular languages FCS

INTUITION

“Regular languages are closed under X” means: if you take regular languages and do X to them, the result is **always** regular again. To prove it you show **how to build a machine** (or regex) for the result from the machines for the inputs.

5.1 The closure properties and their one-line proofs

Operation	Closed?	Construction (the proof)
Union $L_1 \cup L_2$	yes	regex $r_1 + r_2$; or product DFA with final = “ $p \in F_1$ or $q \in F_2$ ”
Concatenation $L_1 L_2$	yes	regex $r_1 r_2$; or NFA: λ -arrows from finals of M_1 to the start of M_2
Star L^*	yes	regex r^* ; or NFA with new start/final and λ -arrow back
Complement $\overline{L}$	yes	take a DFA (complete, with trap state) and swap final and non-final states
Intersection $L_1 \cap L_2$	yes	product DFA with final = $F_1 \times F_2$; or $\overline{\overline{L_1} \cup \overline{L_2}}$ (De Morgan)
Difference $L_1 \setminus L_2$	yes	$L_1 \cap \overline{L_2}$
Reversal L^R	yes	reverse all arrows, swap start and final (add a new start with λ to old finals)
Homomorphism	yes	substitute a string for each letter in the regex

COMPLEMENT NEEDS A DFA

Swapping final/non-final in an **NFA** does *not* give the complement (a string may have one accepting and one rejecting path). Convert to a complete DFA first. That is a classic exam point.

MODEL ANSWER — FCS 2022 Q2 (5 P): “ARGUE THAT REGULAR LANGUAGES ARE CLOSED UNDER COMPLEMENT, INTERSECTION, CONCATENATION AND UNION”

Let L_1, L_2 be regular, with DFAs $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$, $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ and regular expressions r_1, r_2 .

Union: $r_1 + r_2$ is a regular expression with $L(r_1 + r_2) = L_1 \cup L_2$, so $L_1 \cup L_2$ is regular.

Concatenation: likewise $L(r_1 r_2) = L_1 L_2$. **Complement:** let M_1 be complete (every state has a transition on every symbol; add a trap state if needed). The DFA $(Q_1, \Sigma, \delta_1, q_1, Q_1 \setminus F_1)$ ends in exactly the same state as M_1 on every input but accepts exactly when M_1 rejects, so it accepts $\overline{L_1}$.

Intersection: the product DFA $(Q_1 \times Q_2, \Sigma, \delta, (q_1, q_2), F_1 \times F_2)$ with $\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a))$ simulates both machines at once and accepts iff both accept, so it accepts $L_1 \cap L_2$. (Alternatively $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$ by De Morgan, using the previous two.)

5.2 True/false facts (FCS Q1 style — −0.5 per wrong answer!)

Statement	Answer	Why
L CF non-regular, L' regular $\Rightarrow L \cup L'$ is not regular	False	take $L' = \Sigma^*$: $L \cup \Sigma^* = \Sigma^*$ is regular

L CF non-regular, L' regular $\Rightarrow L \cap L'$ is not regular	False	take $L' = \emptyset$: the intersection is $\emptyset$, regular
Every finite language is regular	True	regex = union of its strings
Every subset of a regular language is regular	False	$\{a^n b^n\} \subseteq \Sigma^*$
The union of two non-regular languages is non-regular	False	$L \cup \overline{L} = \Sigma^*$
A finite union of regular languages is regular	True	repeat the union construction
An infinite union of regular languages is regular	False	see 5.4

RECIPE FOR “NECESSARILY” STATEMENTS

“Necessarily” = always. To show **false**, find **one** counterexample — try the extreme languages first: $\emptyset$, $\{\lambda\}$, Σ^* , a finite set.

5.3 Decision properties

For regular languages (given as DFA/NFA/regex) all these are **decidable** (there’s an algorithm that always answers):

- **Membership** $w \in L$? Run the DFA.
- **Emptiness** $L = \emptyset$? Is any final state reachable from the start state? (graph search)
- **Finiteness** — is L infinite? Yes iff there is a **cycle** on some path from the start state to a final state (remove states that can’t reach a final state, then look for a cycle among the rest).
- **Equivalence** $L_1 = L_2$? Check $(L_1 \setminus L_2) \cup (L_2 \setminus L_1) = \emptyset$, or compare minimal DFAs.

5.4 The ☆ question: infinite unions (FCS 2022 Q9)

MODEL ANSWER: “ $L_N = \bigcup_{n \in \mathbb{N}} L_n$ WITH EACH L_n REGULAR — NECESSARILY REGULAR?”

Disprove. Let $N = \mathbb{N}$ and $L_n = \{a^n b^n\}$. Each L_n contains one string, so it is finite and therefore regular. But $\bigcup_{n \in \mathbb{N}} L_n = \{a^n b^n \mid n \geq 0\}$, which is not regular (pumping lemma, Chapter 6). So an infinite union of regular languages need not be regular. (Closure under union only extends to *finite* unions, by induction.)

6. The pumping lemma for regular languages FCS

INTUITION: THE PIGEONHOLE PRINCIPLE

A DFA with n states reading a string of length $\geq n$ visits $n + 1$ states — so some state repeats. The part of the string read between the two visits is a **loop**. You can go around that loop 0 times, twice, 100 times, and the DFA ends in the same place. So every long string in a regular language has a piece y near the start that can be **pumped**.

We use it **backwards**: find a language where pumping breaks the string — then it can't be regular.

THE LEMMA (AS GIVEN IN THE EXAM APPENDIX)

If L is regular, there is $n \geq 1$ such that every $w \in L$ with $|w| \geq n$ can be written $w = xyz$ with $|xy| \leq n$ and $|y| \geq 1$, such that $xy^iz \in L$ for **all** $i \geq 0$.

RECIPE: PROVING L IS NOT REGULAR — IT'S A GAME

1. **Assume** L is regular. Let n be the pumping constant. (The opponent picks n — you don't know it.)
2. **You pick** a string $w \in L$ with $|w| \geq n$, **depending on** n . Choose it so that the first n letters are all the same letter — then y is trapped there.
3. **The opponent** splits $w = xyz$ with $|xy| \leq n$, $|y| \geq 1$. You must handle **every** possible split.
4. **You pick** an i (usually 0 or 2) so that $xy^iz \notin L$.
5. Contradiction $\Rightarrow L$ is not regular.

Example 1: $L = \{a^k b^k \mid k \geq 0\}$

Assume regular, constant n . Pick $w = a^n b^n \in L$, $|w| = 2n \geq n$. Any split with $|xy| \leq n$ has xy inside the first n letters, which are all a 's, so $y = a^j$ with $j \geq 1$. Pump $i = 2$: $xy^2z = a^{n+j} b^n$. It has more a 's than b 's, so it is not in L . Contradiction; L is not regular.

Example 2: $L = \{ww^R\}$ (even palindromes)

Pick $w = a^n b b a^n$. $y = a^j$ inside the first block. $i = 0$: $a^{n-j} b b a^n$ is not a palindrome. Not regular.

Example 3: $L = \{a^{k^2}\}$

Pick $w = a^{n^2}$. $1 \leq |y| = j \leq n$. $i = 2$: length $n^2 + j$, and $n^2 < n^2 + j \leq n^2 + n < (n+1)^2$. Strictly between two consecutive squares, so not a square. Not regular.

Example 4: $\{w \mid |w|_a = |w|_b\}$

Pick $w = a^n b^n$ again (it's in L); same argument as Example 1. *Alternative trick*: if L were regular, then $L \cap a^* b^* = \{a^n b^n\}$ would be regular (closure under $\cap$) — contradiction.

WHERE PEOPLE LOSE POINTS

- Choosing a specific number like $w = a^5b^5$. The string must depend on n .
- Choosing the split yourself (“let $y = a$ ”). You must argue for **all** splits.
- Using the lemma to prove a language **is** regular. It can’t do that (it is only a necessary condition).
- Forgetting to say “contradiction, so L is not regular” at the end.

7. Context-free grammars BOTH

INTUITION: RULES FOR REWRITING

A grammar is a set of **rewrite rules**. Start with the symbol S and keep replacing a variable (capital letter) by the right side of one of its rules, until only letters (terminals) remain. Every string you can reach this way is in the language.

What grammars can do and automata can't: **matching things that are far apart**. The rule $S \rightarrow aSb$ puts an a on the left and a b on the right *at the same time* — that's how you get “the same number of a's and b's”. That's also how programming languages match $($ with $)$ and $\{$ with $\}$.

7.1 Definitions

CONTEXT-FREE GRAMMAR

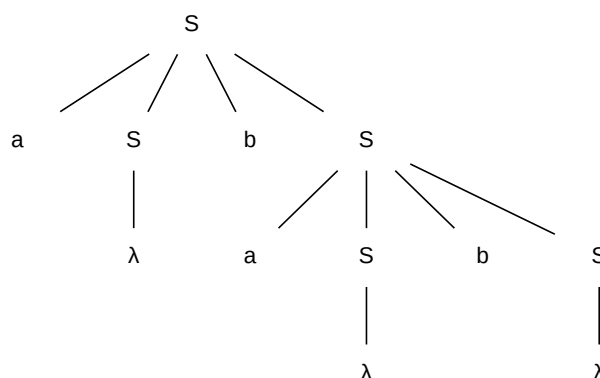
$G = (V, T, S, P)$ (Linz) or $G = (N, \Sigma, R, S)$ (the PL exam): V/N = variables (non-terminals), T/Σ = terminals, S = start variable, P/R = productions of the form $A \rightarrow x$ where A is **one variable** and x is any string of variables and terminals. “Context-free” = you may replace A no matter what surrounds it.

$L(G) = \{w \in T^* \mid S \Rightarrow^* w\}$. A language is **context-free** if some CFG generates it.

- **Derivation**: a sequence of rewrites $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$.
- **Leftmost derivation**: always rewrite the **leftmost** variable. **Rightmost**: always the rightmost.
- **Derivation (parse) tree**: root S ; each rewrite $A \rightarrow x$ gives A children = the symbols of x ; the leaves read left to right give the string.
- **Ambiguous grammar**: some string has **two different parse trees** (equivalently, two different leftmost derivations).

Example: derivations and a tree

Grammar $S \rightarrow aSbS \mid \lambda$, string $abab$. Leftmost derivation: $S \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSbS \Rightarrow ababS \Rightarrow abab$.



7.2 The grammar recipe book

Almost every “construct a CFG” exam question is a combination of these patterns. **All of them were checked by computer on every string up to length 10–12.**

THE CORE TRICK: BUILD FROM THE OUTSIDE IN

If two things must be **equal in number** and they are on **opposite sides**, generate them together with a rule like $S \rightarrow aSb$. Whatever is “in the middle” gets its own variable.

#	Language	Grammar	Idea
R1	$a^n b^n$	$S \rightarrow aSb \mid \lambda$	one a and one b at a time
R2	$a^n b^{2n}$	$S \rightarrow aSbb \mid \lambda$	each a brings two b's
R2'	$a^{2n} b^n$	$S \rightarrow aaSb \mid \lambda$	two a's per b
R3	$a^n b^m, n \leq m$	$S \rightarrow aSb \mid Sb \mid \lambda$	matched pairs + extra b's
R3'	$a^n b^m, n < m$	$S \rightarrow aSb \mid B, B \rightarrow bB \mid b$	at least one extra b in the middle
R4	$a^n b^m, n \neq m$	$S \rightarrow aSb \mid A \mid B, A \rightarrow aA \mid a, B \rightarrow bB \mid b$	$\neq$ means “> or <”: extra a's or extra b's in the middle
R5	$a^n b^m c^{n+m}$	$S \rightarrow aSc \mid B, B \rightarrow bBc \mid \lambda$	a's pair with the outer c's, b's with the inner c's
R5'	$a^n b^{n+m} c^m$	$S \rightarrow AB, A \rightarrow aAb \mid \lambda, B \rightarrow bBc \mid \lambda$	split into $a^n b^n$ followed by $b^m c^m$
R6	$a^i b^j c^k, i = j \text{ or } j = k$	$S \rightarrow AC \mid B; A \rightarrow aAb \mid \lambda, C \rightarrow cC \mid \lambda; B \rightarrow aB \mid D, D \rightarrow bDc \mid \lambda$	“or” = union: $S \rightarrow S_1 \mid S_2$
R7	palindromes	$S \rightarrow aSa \mid bSb \mid a \mid b \mid \lambda$	same letter on both ends
R8	balanced: $ w _a = w _b$, every prefix has $\#a \geq \#b$	$S \rightarrow aSbS \mid \lambda$	first a matched with “its” b
R9	$ w _a = w _b$ (any order)	$S \rightarrow aSbS \mid bSaS \mid \lambda$	first letter is matched with an opposite letter
R10	$ w _a \geq w _b$	$S \rightarrow T \mid TaS, T \rightarrow aTbT \mid bTaT \mid \lambda$	balanced pieces with extra a's between
R11	everything, $(a + b)^*$	$S \rightarrow aS \mid bS \mid \lambda$	regular languages are also CF

```
$ python3 verify/ch07_cfg.py
```

```
[OK] R1 a^n b^n: checked all 8191 strings over {a,b} up to length 12
[OK] R2 a^n b^{2n}: checked all 8191 strings over {a,b} up to length 12
[OK] R2' a^{2n} b^n: checked all 8191 strings over {a,b} up to length 12
[OK] R3 a^n b^m, n <= m: checked all 8191 strings over {a,b} up to length 12
[OK] R3' a^n b^m, n < m: checked all 8191 strings over {a,b} up to length 12
[OK] R4 a^n b^m, n != m: checked all 8191 strings over {a,b} up to length 12
[OK] R5 a^n b^m c^{(n+m)}: checked all 265720 strings over {a,b,c} up to length 11
[OK] R5' a^n b^{(n+m)} c^m: checked all 265720 strings over {a,b,c} up to length 11
[OK] R6 a^i b^j c^k, i=j or j=k: checked all 88573 strings over {a,b,c} up to length 10
[OK] R7 palindromes: checked all 8191 strings over {a,b} up to length 12
[OK] R8 balanced (Dyck): checked all 8191 strings over {a,b} up to length 12
[OK] R9 |w|_a = |w|_b (any order): checked all 8191 strings over {a,b} up to length 12
[OK] R9' alternative: checked all 8191 strings over {a,b} up to length 12
[OK] R10 |w|_a >= |w|_b: checked all 8191 strings over {a,b} up to length 12
[OK] R11 everything (a+b)*: checked all 8191 strings over {a,b} up to length 12
[OK] P1 a^n b^m c^n: checked all 88573 strings over {a,b,c} up to length 10
[OK] P2 m <= n <= 2m: checked all 8191 strings over {a,b} up to length 12
[OK] P3 odd, middle a: checked all 8191 strings over {a,b} up to length 12
```

COMBINING PATTERNS

- **Union** (“... or ...”): $S \rightarrow S_1 \mid S_2$, build S_1, S_2 separately.
- **Concatenation** (“... followed by ...”): $S \rightarrow S_1 S_2$.
- **Repetition**: $S \rightarrow S_1 S \mid \lambda$.
- **“Not equal”**: split into “less than” $\cup$ “greater than”.
- **Anything in between**: a variable like $X \rightarrow aX \mid bX \mid \lambda$.

7.3 The PL exam questions, solved slowly

2023-03-16 / 2023-05-05 Problem 2 (10 p, asked twice)

L_1 = strings over $\{a, b\}$ with **(I)** exactly as many a as b and **(II)** every prefix has at least as many a as b .
Examples: $\varepsilon, ab, aabb, aababbab$.

Understand it first. Read a as “(” and b as “)”. Then (I)+(II) says: the parentheses are **balanced**.
 $aababbab = ((()())())$.

Build it. A non-empty balanced string starts with “(”. That “(” is closed by some “)”. What is between them is balanced, and what comes after is balanced. So $w = a u b v$ with $u, v \in L_1$:

ANSWER

$$S \rightarrow aSbS \mid \varepsilon$$

Check: $S \Rightarrow aSbS \Rightarrow abS \Rightarrow ab$. $aabb$: $S \Rightarrow aSbS \Rightarrow aaSbSbS \Rightarrow^* aabb$. ba cannot be produced since every rule starts with a . Also correct: $S \rightarrow SS \mid aSb \mid \varepsilon$ (but ambiguous).

2023-05-05 Problem 1 (10 p): $L = \{a^n b^m \mid n \neq 2m\}$

THE EXAM CONTRADICTS ITSELF

The formula says $n \neq 2m$ ($\#a \neq 2 \cdot \#b$). The Swedish sentence says “antalet a inte är exakt hälften av antalet b ” ($\#a$ is not half of $\#b$, i.e. $m \neq 2n$). On the exam: **write which one you answer** (“I follow the formula”), or answer both.

Plan (formula reading). The strings with $n = 2m$ are exactly $R2'$: $E \rightarrow aaEb \mid \lambda$. “ $\neq$ ” means $n > 2m$ or $n < 2m$:

- $n > 2m$: an E -string with at least one extra a in front: $A \rightarrow aA \mid aE$.
- $n < 2m$: every b comes with 0, 1 or 2 a 's, and at least one b comes with fewer than 2: $B \rightarrow aaBb \mid aUb \mid Ub$, with $U \rightarrow aaUb \mid aUb \mid Ub \mid \lambda$ (U = “each b with at most 2 a 's”).

ANSWER (FORMULA: $N \neq 2M$)

$$S \rightarrow A \mid B \quad A \rightarrow aA \mid aE \quad E \rightarrow aaEb \mid \lambda \quad B \rightarrow aaBb \mid aUb \mid Ub \quad U \rightarrow aaUb \mid aUb \mid Ub \mid \lambda$$

ANSWER (SENTENCE: $M \neq 2N$)

$$S \rightarrow A \mid B \quad A \rightarrow Ab \mid Eb \quad E \rightarrow aEbb \mid \lambda \quad B \rightarrow aBbb \mid aUb \mid aU \quad U \rightarrow aUbb \mid aUb \mid aU \mid \lambda$$

\$ both readings + Problem 2, every string up to length 14

[OK] 2023-05-05 P1, formula reading $a^n b^m$, $n \neq 2m$: checked all 32767 strings over $\{a,b\}$ up to length 14

[OK] 2023-05-05 P1, sentence reading $a^n b^m$, $m \neq 2n$: checked all 32767 strings over $\{a,b\}$ up to length 14

[OK] L1 with $S \rightarrow aSbS \mid \varepsilon$: checked all 32767 strings over $\{a,b\}$ up to length 14

[OK] L1 with $S \rightarrow SS \mid aSb \mid \varepsilon$: checked all 32767 strings over $\{a,b\}$ up to length 14

2023-03-16 Problem 1 (10 p): which strings does G generate?

$S \rightarrow SB \mid aA \mid cCb$, $A \rightarrow aAb \mid \varepsilon$, $B \rightarrow BB \mid C$, $C \rightarrow \varepsilon$. Candidates: ε , abcb, aabcb, cbab, abc, acb, aaabbbbccbb, aaabbbbccbbcc, ccbbaab.

RECIPE: FIRST FIND WHAT EACH VARIABLE GENERATES, BOTTOM-UP

1. $C \rightarrow \varepsilon$ only: C generates only ε .
2. $B \rightarrow BB \mid C$: only ever ε .
3. $A \rightarrow aAb \mid \varepsilon$: $a^k b^k$.
4. $S \rightarrow SB$ adds nothing (B is ε). $S \rightarrow aA$ gives $a^{k+1} b^k$. $S \rightarrow cCb$ gives cb .

So $L(G) = \{a^{k+1} b^k \mid k \geq 0\} \cup \{cb\} = \{a, aab, aaabb, \dots\} \cup \{cb\}$.

ANSWER

None of the nine strings is generated. Every string of $L(G)$ is either cb or a 's followed by one fewer b 's — no candidate has that form. (Verified by a parser.)

\$ membership check

2023-03-16 Problem 1 — membership of each string:

A. ε	no
B. abcb	no
C. aabcb	no
D. cbab	no
E. abc	no
F. acb	no
G. aaabbbbccbb	no
H. aaabbbbccbbcc	no
I. ccbbaab	no

[OK] $L(G) = \{a^{k+1} b^k : k \geq 0\} \cup \{cb\}$: checked all 29524 strings over $\{a,b,c\}$ up to length 9

7.4 Ambiguity

$E \rightarrow E + E \mid E * E$ is ambiguous: $a + a * a$ has two trees, one meaning $a + (a * a)$, one $(a + a) * a$. A compiler needs **one** meaning, so we rewrite the grammar with levels (Chapter 8). Another classic is the **dangling else**: in `if c1 then if c2 then s1 else s2`, which `if` owns the `else`? (Rule in most languages: the nearest one.)

TO SHOW A GRAMMAR IS AMBIGUOUS

Give **one string** and **two different leftmost derivations** (or two trees) for it. That's a complete answer.

ALWAYS TEST YOUR GRAMMAR BEFORE MOVING ON (1 MINUTE)

1. Can it produce the **shortest** member (often ϵ)? Should it?
2. Derive 2 members from the examples.
3. Try to produce 1 non-member — make sure you can't.

PRACTICE

1. $\{a^n b^m c^n \mid n, m \geq 0\}$
2. $\{a^n b^m \mid m \leq n \leq 2m\}$
3. $\{w \in \{a, b\}^* \mid w \text{ has odd length and middle letter } a\}$
4. Show $S \rightarrow SS \mid aSb \mid \lambda$ is ambiguous.

SOLUTIONS

1. $S \rightarrow aSc \mid B, B \rightarrow bB \mid \lambda$.
2. $S \rightarrow aSb \mid aaSb \mid \lambda$ (each b takes 1 or 2 a's).
3. $S \rightarrow XSX \mid a, X \rightarrow a \mid b$.
4. $\lambda: S \Rightarrow \lambda$ and $S \Rightarrow SS \Rightarrow S \Rightarrow \lambda$; or $abab$ via $S \Rightarrow SS$ split in different ways (e.g. $SS \Rightarrow SSS$ first).

8. Precedence and associativity PL

INTUITION

Without parentheses, $2 + 3 * 4$ could mean 20 or 14. The language must decide. **Precedence:** which operator wins between *different* operators ($*$ beats $+$). **Associativity:** which side wins between *equal* operators ($10 - 4 - 3$ is $(10 - 4) - 3$).

8.1 Common rules

Level (high → low)	Operators	Associativity
1	$^$ / $**$ (power)	right: $2^3^2 = 2^3^2 = 512$
2	unary $-$, $!$	right
3	$*$ / $\%$	left
4	$+$ -	left: $10-4-3 = 3$
5	$<$ $==$...	often non-associative
6	$\&\&$, then $ $	left
7	$=$ (assignment)	right: $a = b = 0$

In Haskell you can even declare them: `infixl 6 +`, `infixl 7 *`, `infixr 8 ^`, `infixr 5 :`, `infixr 5 ++`. Function application binds tighter than any operator: `f x + 1 = (f x) + 1`.

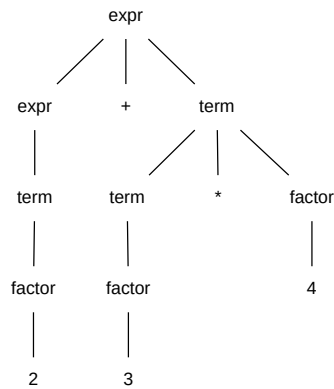
8.2 How a grammar encodes them

RECIPE: ONE VARIABLE PER PRECEDENCE LEVEL

```
expr  → expr + term  | expr - term  | term          (lowest precedence, left-assoc)
term  → term * factor | term / factor | factor
factor → base ^ factor | base          (right-assoc: RIGHT recursion)
base  → number | ( expr )              (highest: parentheses)
```

- **Lower in the grammar = binds tighter:** a `term` is built before it can be added.
- **Left recursion** (`expr → expr + term`) grows the tree to the left $\Rightarrow$ **left associative**.
- **Right recursion** (`factor → base ^ factor`) $\Rightarrow$ **right associative**.

This grammar is unambiguous: $2 + 3 * 4$ has exactly one tree, with $*$ below $+$:



A working parser built exactly this way (in Haskell) is in Chapter 22; it really turns `10 - 4 - 3` into `(10 - 4) - 3`.

MODEL ANSWER (6 P): “BRIEFLY DESCRIBE PRECEDENCE AND ASSOCIATIVITY. HOW ARE THEY RELATED?”

Precedence decides which of two **different** operators binds tighter when there are no parentheses: `*` has higher precedence than `+`, so `2 + 3 * 4 = 2 + (3 * 4) = 14`. **Associativity** decides how operators of the **same** precedence level are grouped: `-` is left-associative, so `10 - 4 - 3 = (10 - 4) - 3 = 3`; exponentiation and assignment are usually right-associative (`2^3^2 = 2^(3^2)`, `a = b = 0`).

Relation: both are rules that remove the **ambiguity** of expressions without parentheses and thereby fix the shape of the syntax tree (and so the result). Precedence is used between different levels; associativity only matters when operators have the **same** precedence. In a grammar they are encoded by one non-terminal per precedence level (lower levels deeper in the tree bind tighter) and by the direction of recursion (left recursion $\Rightarrow$ left associative, right recursion $\Rightarrow$ right associative). Parentheses override both.

9. Pushdown automata FCS

INTUITION: A FINITE AUTOMATON PLUS A STACK

A DFA can't count without bound. Give it a **stack** (a pile of plates: you can only add or remove on top) and it can: for $a^n b^n$, push one plate per a , pop one per b , and accept if the pile is back to the bottom exactly when the input ends. PDAs recognise **exactly the context-free languages** — the stack is what a parser uses to match brackets.

DEFINITION (EXAM APPENDIX): NPDA

$M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$: states, input alphabet, stack alphabet Γ , transition function $\delta : Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \rightarrow \text{finite subsets of } Q \times \Gamma^*$, start state, stack start symbol z , final states.

$(p, x) \in \delta(q, a, s)$ means: in state q , reading a (or nothing if $a = \lambda$), with s on top of the stack: go to p and **replace s by the string x** (leftmost symbol of x becomes the new top). So: **pop** = replace by λ ; **push A** = replace s by As ; **no change** = replace s by s .

Accepts w if some computation reads all of w and ends in a final state.

Example: $\{a^n b^n \mid n \geq 0\}$

Transition	Meaning
$\delta(q_0, a, z) = \{(q_0, Az)\}, \delta(q_0, a, A) = \{(q_0, AA)\}$	push an A for each a
$\delta(q_0, b, A) = \{(q_1, \lambda)\}, \delta(q_1, b, A) = \{(q_1, \lambda)\}$	pop an A for each b ; once b 's start, no more a 's
$\delta(q_0, \lambda, z) = \{(q_2, z)\}, \delta(q_1, \lambda, z) = \{(q_2, z)\}$	stack back at bottom $\Rightarrow$ go to final q_2

$F = \{q_2\}$. Trace $aabb$: $(q_0, aabb, z) \vdash (q_0, abb, Az) \vdash (q_0, bb, AAz) \vdash (q_1, b, Az) \vdash (q_1, \lambda, z) \vdash (q_2, \lambda, z)$ — accept. These triples (state, remaining input, stack) are **instantaneous descriptions**.

Example: even palindromes ww^R — uses nondeterminism

Push every letter; at some point **guess** “this is the middle” (a λ -move to a pop state); then pop and compare each letter with the top. A deterministic machine can't know where the middle is — this language is CF but **not** deterministic CF.

9.1 CFG $\rightarrow$ PDA

CONSTRUCTION (THE PROOF THAT EVERY CFL HAS A PDA)

One working state. Push S . Then repeat: if the top is a **variable** A , replace it (λ -move) by the right side of some rule $A \rightarrow x$ (guess which); if the top is a **terminal**, it must match the next input letter — pop it and read. Accept when input is used up and the stack is empty (at z). The PDA simulates a leftmost derivation.

9.2 Deterministic PDAs

- DPDA: at most one possible move in every situation (and no choice between a λ -move and a reading move).

- **DCFL** = languages of DPDAs. Regular $\subset$ DCFL $\subset$ CFL, both strict: $a^n b^n$ is DCFL but not regular; ww^R is CFL but not DCFL.
- DCFLs are closed under **complement**; CFLs are not. Programming-language parsers (LR) work with DCFL-style grammars because they must be fast (linear time).
- Unlike finite automata, **NPDAs are strictly more powerful than DPDAs**.

10. Chomsky normal form and the CYK algorithm BOTH

Part A — Chomsky normal form

THE IDEA IN PLAIN WORDS

A grammar in CNF is a grammar where **every rule is one of only two boring shapes**:

$$A \rightarrow BC \text{ (exactly two variables) or } A \rightarrow a \text{ (exactly one terminal).}$$

Why would anyone want that? Because then every derivation tree is a **binary tree**, and a string of length n is always derived in exactly $2n - 1$ steps. That makes algorithms like CYK (Part B) possible. Converting to CNF does **not change the language** (except that λ disappears) — it only rewrites the rules into boring shapes.

DEFINITION (WRITE THIS ON THE EXAM)

A context-free grammar is in **Chomsky normal form** if every production has the form

$$A \rightarrow BC \quad \text{or} \quad A \rightarrow a$$

where A, B, C are variables and a is a terminal. Every CFG G with $\lambda \notin L(G)$ has an equivalent grammar in CNF. (If $\lambda \in L(G)$, the CNF grammar generates $L(G) \setminus \{\lambda\}$.)

Checking whether a grammar is in CNF

Go through the rules one by one and mark anything that is not of those two shapes:

Rule	CNF?	Why
$S \rightarrow AB$	✓	two variables
$A \rightarrow a$	✓	one terminal
$S \rightarrow aB$	✗	terminal mixed with a variable
$S \rightarrow ABC$	✗	three variables (too long)
$A \rightarrow B$	✗	a <i>unit</i> production (single variable)
$A \rightarrow \lambda$	✗	a λ -production
$A \rightarrow ab$	✗	two terminals

10.A1 The recipe (Linz order: $\lambda \rightarrow$ units $\rightarrow$ useless $\rightarrow$ shape)

RECIPE: CONVERTING TO CNF — ALWAYS IN THIS ORDER

1. Remove λ -productions.

- Find the **nullable** variables: A is nullable if $A \rightarrow \lambda$, or $A \rightarrow B_1 \dots B_k$ where all B_i are nullable. Repeat until nothing new.
- For every rule, add **all versions where you leave out any combination of nullable variables** (but never add $A \rightarrow \lambda$).
- Delete all $A \rightarrow \lambda$.

2. Remove unit productions ($A \rightarrow B$).

- For every A , find all B with $A \Rightarrow^* B$ using *only* unit rules.
- Give A a copy of every **non-unit** rule of each such B .
- Delete all unit rules (including $A \rightarrow A$).

3. Remove useless variables, in this order:

- non-generating**: variables that can never finish into a terminal string (e.g. $C \rightarrow cC$ only);
- non-reachable**: variables that can't be reached from S .

Delete every rule that mentions them.

4. Fix the shapes.

- In every rule whose right side has length ≥ 2 , replace each terminal a by a new variable T_a and add $T_a \rightarrow a$.
- Break every right side with 3 or more variables into a chain: $A \rightarrow C_1 C_2 C_3 \dots C_n$ becomes $A \rightarrow C_1 D_1, D_1 \rightarrow C_2 D_2, \dots, D_{n-2} \rightarrow C_{n-1} C_n$.

WHY THE ORDER MATTERS

Removing λ can **create** unit rules (e.g. $S \rightarrow aSb \mid SS$ with nullable S gives $S \rightarrow S$). Removing units can **create** useless variables (a variable nobody uses any more). So: λ first, units second, useless third, shapes last. If you do it in another order you will have to repeat steps.

10.A2 Worked example A — the exam's own grammar $S \rightarrow aSbS \mid \lambda$

This is the grammar for the PL-exam language “equally many a and b , every prefix has at least as many a as b ”. Convert it to CNF.

Step 1 — λ -productions. Nullable: S (because $S \rightarrow \lambda$). The rule $S \rightarrow aSbS$ contains two occurrences of S , so we write all four combinations of keeping/dropping them:

keep 1st S?	keep 2nd S?	new rule
yes	yes	$S \rightarrow aSbS$
no	yes	$S \rightarrow abS$
yes	no	$S \rightarrow aSb$
no	no	$S \rightarrow ab$

Delete $S \rightarrow \lambda$. Result: $S \rightarrow aSbS \mid abS \mid aSb \mid ab$.

Step 2 — unit productions. None (no rule of the form $S \rightarrow X$). **Step 3 — useless.** None.

Step 4a — terminals. Introduce $A \rightarrow a$ and $B \rightarrow b$: $S \rightarrow ASBS \mid ABS \mid ASB \mid AB$.

Step 4b — long right sides.

- $S \rightarrow ASBS$ (4 symbols) $\rightarrow S \rightarrow AX$, $X \rightarrow SW$, $W \rightarrow BS$.
- $S \rightarrow ABS$ (3 symbols) $\rightarrow S \rightarrow AY$, $Y \rightarrow BS$.
- $S \rightarrow ASB$ (3 symbols) $\rightarrow S \rightarrow AZ$, $Z \rightarrow SB$.
- $S \rightarrow AB$ is already fine.

FINAL ANSWER

$S \rightarrow AX \mid AY \mid AZ \mid AB$ $X \rightarrow SW$ $W \rightarrow BS$ $Y \rightarrow BS$ $Z \rightarrow SB$ $A \rightarrow a$ $B \rightarrow b$

Since λ was in the original language, the CNF grammar generates the same language minus λ . (You may reuse W instead of introducing Y — both are correct.)

10.A3 Worked example B — every step is needed

G : $S \rightarrow ASA \mid aB$ $A \rightarrow B \mid S$ $B \rightarrow b \mid \lambda$

Step 1 — λ -productions.

- Nullable: B (since $B \rightarrow \lambda$); then A (since $A \rightarrow B$ and B is nullable). Is S ? $S \rightarrow ASA$ needs S itself nullable, $S \rightarrow aB$ has a terminal. So **no**. Nullable = $\{A, B\}$.
- $S \rightarrow ASA$: the two A 's may each be dropped: ASA , SA , AS , S .
- $S \rightarrow aB$: drop B : aB , a .
- $A \rightarrow B$: dropping B would give $A \rightarrow \lambda$ — never add that. $A \rightarrow S$ unchanged.
- Delete $B \rightarrow \lambda$.

Result: $S \rightarrow ASA \mid SA \mid AS \mid S \mid aB \mid a$ $A \rightarrow B \mid S$ $B \rightarrow b$

Step 2 — unit productions. The unit rules are $S \rightarrow S$, $A \rightarrow B$, $A \rightarrow S$.

- $S \rightarrow S$: just delete it.
- $A \Rightarrow^* B$ and $A \Rightarrow^* S$. So A gets all non-unit rules of B (b) and of S (ASA , SA , AS , aB , a).

Result: $S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$ $A \rightarrow b \mid ASA \mid SA \mid AS \mid aB \mid a$ $B \rightarrow b$

Step 3 — useless. Everything generates a terminal string and is reachable. Nothing to remove.

Step 4. $aB \rightarrow UB$ with $U \rightarrow a$. $ASA \rightarrow AX$ with $X \rightarrow SA$.

FINAL ANSWER

$S \rightarrow AX \mid SA \mid AS \mid UB \mid a$
 $A \rightarrow b \mid AX \mid SA \mid AS \mid UB \mid a$
 $B \rightarrow b$ $U \rightarrow a$ $X \rightarrow SA$

10.A4 Worked example C — unit chains and useless variables

G : $S \rightarrow Aa \mid B \mid C$ $A \rightarrow a \mid bA$ $B \rightarrow A \mid bb$ $C \rightarrow cC$

1. λ : no λ -rules. Skip.

2. **Units:** $S \rightarrow B$, $S \rightarrow C$, $B \rightarrow A$. Unit reachability: $S \Rightarrow^* B, A, C$; $B \Rightarrow^* A$. So S gets bb (from B), a, bA (from A), cC (from C); B gets a, bA . Now: $S \rightarrow Aa \mid bb \mid a \mid bA \mid cC$, $A \rightarrow a \mid bA$, $B \rightarrow bb \mid a \mid bA$, $C \rightarrow$

cC .

3. **Useless:** (a) C is **not generating** – $C \rightarrow cC$ never ends. Delete C and $S \rightarrow cC$. (b) B is now **unreachable** – nobody mentions B . Delete it. Now: $S \rightarrow Aa \mid bb \mid a \mid bA$, $A \rightarrow a \mid bA$.
4. **Shapes:** $V \rightarrow a$ and $T \rightarrow b$: $S \rightarrow AV \mid TT \mid a \mid TA$, $A \rightarrow a \mid TA$.

FINAL ANSWER

$S \rightarrow AV \mid TT \mid a \mid TA \quad A \rightarrow a \mid TA \quad V \rightarrow a \quad T \rightarrow b$

Note: you cannot reuse A for “ $\rightarrow a$ ” here, because A also has the rule $A \rightarrow TA$; a new variable that *only* produces a is required.

THE MISTAKES THAT COST POINTS

- Dropping **only one** combination of nullable symbols. With two nullable symbols in a rule there are up to 4 versions (with three, up to 8). Make the little table.
- Adding $A \rightarrow \lambda$ back by dropping everything.
- Forgetting that a variable is nullable *indirectly* ($A \rightarrow B$, $B \rightarrow \lambda$).
- Removing a unit rule without copying the rules over (then the language shrinks).
- Replacing a terminal by a variable that has other rules too (see example C).
- Not saying anything about λ . One sentence: “ $\lambda \in L(G)$, so the CNF grammar generates $L(G) - \{\lambda\}$.”

Proof that these answers are right

Each grammar below was compared with the original on every non-empty string (up to the length shown) using a general parser, and checked to be in CNF:

```
$ python3 verify/pack_cnf.py --check
```

```
[OK] Ex1 after removing  $\lambda$ :  $L(\text{CNF}) = L(G) - \{\lambda\}$  on all strings up to length 10; in CNF: False
[OK] Ex1 after removing units:  $L(\text{CNF}) = L(G) - \{\lambda\}$  on all strings up to length 10; in CNF:
False
[OK] Ex1 final CNF:  $L(\text{CNF}) = L(G) - \{\lambda\}$  on all strings up to length 10; in CNF: True
[OK] Ex2 after removing  $\lambda$ :  $L(\text{CNF}) = L(G) - \{\lambda\}$  on all strings up to length 12; in CNF: False
[OK] Ex2 final CNF:  $L(\text{CNF}) = L(G) - \{\lambda\}$  on all strings up to length 12; in CNF: True
[OK] Ex3 final CNF:  $L(\text{CNF}) = L(G) - \{\lambda\}$  on all strings up to length 9; in CNF: True
```

Part B — The CYK algorithm

THE IDEA IN PLAIN WORDS

Question: *can the grammar produce the string w ?* CYK answers it bottom-up, like building a pyramid:

- First, for every **single letter**: which variables can produce it? (Look for rules $A \rightarrow a$.)
- Then, for every piece of **length 2**: which variables can produce it? Because the grammar is in CNF, a variable produces a piece only through a rule $A \rightarrow BC$: B makes the left part, C makes the right part. So **try every place to cut the piece in two**, and look up the two halves in the rows you already have.
- Then length 3, 4, ... up to the whole string.
- At the top: if the **start symbol** can produce the whole string, $w \in L(G)$.

That's all. The grammar **must be in CNF** first (that's why CNF and CYK always come together).

DEFINITION (LINZ NOTATION — WRITE THIS)

Let $w = a_1 a_2 \cdots a_n$. Define V_{ij} = the set of variables that derive the substring $a_i \cdots a_j$. Then

$$V_{ii} = \{A \mid A \rightarrow a_i\}$$
$$V_{ij} = \bigcup_{k=i}^{j-1} \{A \mid A \rightarrow BC, B \in V_{ik}, C \in V_{k+1,j}\}$$

and $w \in L(G) \iff S \in V_{1n}$. The running time is $O(n^3)$.

RECIPE FOR THE EXAM

1. Check that the grammar is in CNF (say so explicitly — the exam often asks).
2. Draw a triangle: row 1 has n cells (one per letter), row 2 has $n - 1$, ..., the top row has one cell. Label each cell with its substring — this avoids mistakes.
3. Row 1: for each letter, write the variables with a rule $X \rightarrow$ that letter.
4. Row ℓ : for each substring of length ℓ , for **every** cut into (left, right): take the pairs $B \in \text{cell}(\text{left})$, $C \in \text{cell}(\text{right})$, and add every A with a rule $A \rightarrow BC$. Write $\emptyset$ if nothing.
5. Top cell: contains S ? Then write the conclusion sentence (below).

THE “VERY BRIEFLY EXPLAIN WHAT THE RESULT MEANS” SENTENCE

“Cell V_{ij} contains exactly the variables that can derive the substring from position i to j . Since the start symbol S is in the top cell V_{1n} , $S \Rightarrow^* w$, so the string **is** in the language of the grammar.” (If $S \notin V_{1n}$: “...so the string is **not** in the language.”)

Bonus sentence for full points: “Following the entries back down (which cut produced S) gives a derivation tree.”

10.B1 Worked example 1 — the real exam question (abcab)

Grammar: $S \rightarrow SS \mid XY \mid XS', S' \rightarrow SY, X \rightarrow a, Y \rightarrow BC \mid BB \mid b, B \rightarrow b, C \rightarrow CC \mid c$.

Is it in CNF? Yes: every rule is either two variables ($SS, XY, XS', SY, BC, BB, CC$) or one terminal (a, b, c). (S' is just a variable name.) So the answer is “Yes”.

Here is every cell with every cut tried (generated by the checker program — this is exactly the work you do on paper):

\$ CYK on abcab, every cut

V11 ('a'): variables with a rule $\rightarrow$ a: {X}
V22 ('b'): variables with a rule $\rightarrow$ b: {B,Y}
V33 ('c'): variables with a rule $\rightarrow$ c: {C}
V44 ('a'): variables with a rule $\rightarrow$ a: {X}
V55 ('b'): variables with a rule $\rightarrow$ b: {B,Y}

V12 ('ab'): a|b: {X}·{B,Y} $\rightarrow$ {S} $\Rightarrow$ {S}
V23 ('bc'): b|c: {B,Y}·{C} $\rightarrow$ {Y} $\Rightarrow$ {Y}
V34 ('ca'): c|a: {C}·{X} $\rightarrow$ $\emptyset$ $\Rightarrow$ $\emptyset$
V45 ('ab'): a|b: {X}·{B,Y} $\rightarrow$ {S} $\Rightarrow$ {S}

V13 ('abc'): a|bc: {X}·{Y} $\rightarrow$ {S}; ab|c: {S}·{C} $\rightarrow$ $\emptyset$ $\Rightarrow$ {S}
V24 ('bca'): b|ca: {B,Y}· $\emptyset$ $\rightarrow$ $\emptyset$; bc|a: {Y}·{X} $\rightarrow$ $\emptyset$ $\Rightarrow$ $\emptyset$
V35 ('cab'): c|ab: {C}·{S} $\rightarrow$ $\emptyset$; ca|b: $\emptyset$ ·{B,Y} $\rightarrow$ $\emptyset$ $\Rightarrow$ $\emptyset$

V14 ('abca'): a|bca: {X}· $\emptyset$ $\rightarrow$ $\emptyset$; ab|ca: {S}· $\emptyset$ $\rightarrow$ $\emptyset$; abc|a: {S}·{X} $\rightarrow$ $\emptyset$ $\Rightarrow$ $\emptyset$
V25 ('bcab'): b|cab: {B,Y}· $\emptyset$ $\rightarrow$ $\emptyset$; bc|ab: {Y}·{S} $\rightarrow$ $\emptyset$; bca|b: $\emptyset$ ·{B,Y} $\rightarrow$ $\emptyset$ $\Rightarrow$ $\emptyset$

V15 ('abcab'): a|bcab: {X}· $\emptyset$ $\rightarrow$ $\emptyset$; ab|cab: {S}· $\emptyset$ $\rightarrow$ $\emptyset$; abc|ab: {S}·{S} $\rightarrow$ {S}; abca|b: $\emptyset$ ·{B,Y} $\rightarrow$ $\emptyset$ $\Rightarrow$ {S}

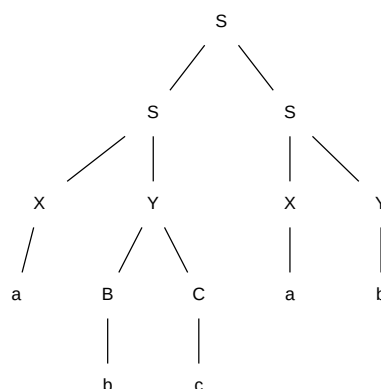
Start symbol $S \in V_{15} \Rightarrow$ 'abcab' IS in $L(G)$

How to read one line: $abc|ab: \{S\} \cdot \{S\} \rightarrow \{S\}$ means “cut *abcab* into *abc* and *ab*; *S* makes *abc*, *S* makes *ab*; the rule $S \rightarrow SS$ combines them, so *S* makes *abcab*”.

The table as you would draw it on the exam (bottom row = single letters):

len 5	$V_{15} = \{S\}$				
len 4	$V_{14} = \emptyset$	$V_{25} = \emptyset$			
len 3	$V_{13} = \{S\}$	$V_{24} = \emptyset$	$V_{35} = \emptyset$		
len 2	$V_{12} = \{S\}$	$V_{23} = \{Y\}$	$V_{34} = \emptyset$	$V_{45} = \{S\}$	
len 1	$V_{11} = \{X\}$	$V_{22} = \{B, Y\}$	$V_{33} = \{C\}$	$V_{44} = \{X\}$	$V_{55} = \{B, Y\}$
	a	b	c	a	b

Result: $S \in V_{15}$, so $abcab \in L(G)$. Reading back: $S \rightarrow SS$ with the first *S* deriving *abc* (via $S \rightarrow XY$, $X \rightarrow a$, $Y \rightarrow BC \rightarrow bc$) and the second deriving *ab* (via $S \rightarrow XY$, $X \rightarrow a$, $Y \rightarrow b$).



Derivation tree recovered from the table

10.B2 Worked example 2 — a string that is *rejected* (length 5)

Grammar: the CNF grammar from Part A (for the language “balanced a/b, prefixes have $\geq$ as many a as b”): $S \rightarrow AX \mid AY \mid AZ \mid AB, X \rightarrow SW, W \rightarrow BS, Y \rightarrow BS, Z \rightarrow SB, A \rightarrow a, B \rightarrow b$. String: aabab.

\$ CYK on aabab

V11 ('a'): variables with a rule $\rightarrow a$: {A}

V22 ('a'): variables with a rule $\rightarrow a$: {A}

V33 ('b'): variables with a rule $\rightarrow b$: {B}

V44 ('a'): variables with a rule $\rightarrow a$: {A}

V55 ('b'): variables with a rule $\rightarrow b$: {B}

V12 ('aa'): $a|a: \{A\} \cdot \{A\} \rightarrow \emptyset \Rightarrow \emptyset$

V23 ('ab'): $a|b: \{A\} \cdot \{B\} \rightarrow \{S\} \Rightarrow \{S\}$

V34 ('ba'): $b|a: \{B\} \cdot \{A\} \rightarrow \emptyset \Rightarrow \emptyset$

V45 ('ab'): $a|b: \{A\} \cdot \{B\} \rightarrow \{S\} \Rightarrow \{S\}$

V13 ('aab'): $a|ab: \{A\} \cdot \{S\} \rightarrow \emptyset$; $aa|b: \emptyset \cdot \{B\} \rightarrow \emptyset \Rightarrow \emptyset$

V24 ('aba'): $a|ba: \{A\} \cdot \emptyset \rightarrow \emptyset$; $ab|a: \{S\} \cdot \{A\} \rightarrow \emptyset \Rightarrow \emptyset$

V35 ('bab'): $b|ab: \{B\} \cdot \{S\} \rightarrow \{W, Y\}$; $ba|b: \emptyset \cdot \{B\} \rightarrow \emptyset \Rightarrow \{W, Y\}$

V14 ('aaba'): $a|aba: \{A\} \cdot \emptyset \rightarrow \emptyset$; $aa|ba: \emptyset \cdot \emptyset \rightarrow \emptyset$; $aab|a: \emptyset \cdot \{A\} \rightarrow \emptyset \Rightarrow \emptyset$

V25 ('abab'): $a|bab: \{A\} \cdot \{W, Y\} \rightarrow \{S\}$; $ab|ab: \{S\} \cdot \{S\} \rightarrow \emptyset$; $aba|b: \emptyset \cdot \{B\} \rightarrow \emptyset \Rightarrow \{S\}$

V15 ('aabab'): $a|aabab: \{A\} \cdot \{S\} \rightarrow \emptyset$; $aa|bab: \emptyset \cdot \{W, Y\} \rightarrow \emptyset$; $aab|ab: \emptyset \cdot \{S\} \rightarrow \emptyset$; $aaba|b: \emptyset \cdot \{B\} \rightarrow \emptyset \Rightarrow \emptyset$

Start symbol $S \notin V_{15} \Rightarrow$ 'aabab' is NOT in $L(G)$

$S \notin V_{15} = \emptyset$, so aabab is **not** in the language. Sanity check: it has 3 a's and 2 b's, so it can't be balanced.

✓ (Always do such a sanity check if you can!)

10.B3 Worked example 3 — accepted, length 6

\$ CYK on abaabb

V11 ('a'): variables with a rule $\rightarrow a: \{A\}$

V22 ('b'): variables with a rule $\rightarrow b: \{B\}$

V33 ('a'): variables with a rule $\rightarrow a: \{A\}$

V44 ('a'): variables with a rule $\rightarrow a: \{A\}$

V55 ('b'): variables with a rule $\rightarrow b: \{B\}$

V66 ('b'): variables with a rule $\rightarrow b: \{B\}$

V12 ('ab'): $a|b: \{A\} \cdot \{B\} \rightarrow \{S\} \Rightarrow \{S\}$

V23 ('ba'): $b|a: \{B\} \cdot \{A\} \rightarrow \emptyset \Rightarrow \emptyset$

V34 ('aa'): $a|a: \{A\} \cdot \{A\} \rightarrow \emptyset \Rightarrow \emptyset$

V45 ('ab'): $a|b: \{A\} \cdot \{B\} \rightarrow \{S\} \Rightarrow \{S\}$

V56 ('bb'): $b|b: \{B\} \cdot \{B\} \rightarrow \emptyset \Rightarrow \emptyset$

V13 ('aba'): $a|ba: \{A\} \cdot \emptyset \rightarrow \emptyset$; $ab|a: \{S\} \cdot \{A\} \rightarrow \emptyset \Rightarrow \emptyset$

V24 ('baa'): $b|aa: \{B\} \cdot \emptyset \rightarrow \emptyset$; $ba|a: \emptyset \cdot \{A\} \rightarrow \emptyset \Rightarrow \emptyset$

V35 ('aab'): $a|ab: \{A\} \cdot \{S\} \rightarrow \emptyset$; $aa|b: \emptyset \cdot \{B\} \rightarrow \emptyset \Rightarrow \emptyset$

V46 ('abb'): $a|bb: \{A\} \cdot \emptyset \rightarrow \emptyset$; $ab|b: \{S\} \cdot \{B\} \rightarrow \{Z\} \Rightarrow \{Z\}$

V14 ('abaa'): $a|baa: \{A\} \cdot \emptyset \rightarrow \emptyset$; $ab|aa: \{S\} \cdot \emptyset \rightarrow \emptyset$; $aba|a: \emptyset \cdot \{A\} \rightarrow \emptyset \Rightarrow \emptyset$

V25 ('baab'): $b|aab: \{B\} \cdot \emptyset \rightarrow \emptyset$; $ba|ab: \emptyset \cdot \{S\} \rightarrow \emptyset$; $baa|b: \emptyset \cdot \{B\} \rightarrow \emptyset \Rightarrow \emptyset$

V36 ('aabb'): $a|abb: \{A\} \cdot \{Z\} \rightarrow \{S\}$; $aa|bb: \emptyset \cdot \emptyset \rightarrow \emptyset$; $aab|b: \emptyset \cdot \{B\} \rightarrow \emptyset \Rightarrow \{S\}$

V15 ('abaab'): $a|baab: \{A\} \cdot \emptyset \rightarrow \emptyset$; $ab|aab: \{S\} \cdot \emptyset \rightarrow \emptyset$; $aba|ab: \emptyset \cdot \{S\} \rightarrow \emptyset$; $abaa|b: \emptyset \cdot \{B\} \rightarrow \emptyset \Rightarrow \emptyset$

V26 ('baabb'): $b|aabb: \{B\} \cdot \{S\} \rightarrow \{W, Y\}$; $ba|abb: \emptyset \cdot \{Z\} \rightarrow \emptyset$; $baa|bb: \emptyset \cdot \emptyset \rightarrow \emptyset$; $baab|b: \emptyset \cdot \{B\} \rightarrow \emptyset \Rightarrow \{W, Y\}$

V16 ('abaabb'): $a|baabb: \{A\} \cdot \{W, Y\} \rightarrow \{S\}$; $ab|aabb: \{S\} \cdot \{S\} \rightarrow \emptyset$; $aba|abb: \emptyset \cdot \{Z\} \rightarrow \emptyset$; $abaa|bb: \emptyset \cdot \emptyset \rightarrow \emptyset$; $abaab|b: \emptyset \cdot \{B\} \rightarrow \emptyset \Rightarrow \{S\}$

Start symbol $S \in V_{16} \Rightarrow$ 'abaabb' IS in $L(G)$

$S \in V_{16}$: abaabb is in the language (it is $ab \cdot aabb$).

THE MISTAKES THAT COST POINTS

- **Not trying every cut.** A cell of length ℓ has $\ell - 1$ cuts. Example 1's top cell has 4 cuts, and only the third one works.
- Order matters: $A \rightarrow BC$ needs B on the **left** part and C on the **right** part.
- Forgetting that a cell can hold **several** variables (like $\{B, Y\}$) — try all of them.
- Not writing the conclusion sentence. "Briefly explain what the result means" is worth points on its own.
- Running CYK on a grammar that is not in CNF — convert first (Part A).

11. Pumping lemma for CFLs, and closure FCS

INTUITION

In a very tall parse tree, some variable must repeat on one path (finitely many variables — pigeonhole again). The subtree between the two occurrences can be repeated or removed. That pumps **two** pieces u and v **at the same time** (one on each side of the middle y). This is why CFLs can match **two** counts ($a^n b^n$) but not **three** ($a^n b^n c^n$).

THE LEMMA (EXAM APPENDIX)

If L is (infinite) context-free, there is n such that every $w \in L$ with $|w| \geq n$ can be written $w = xuyvz$ with $|uyv| \leq n$, $|uv| \geq 1$, such that $xu^i y v^i z \in L$ for all $i \geq 0$.

RECIPE: PROVING L IS NOT CONTEXT-FREE

1. Assume L is CF with constant n .
2. Choose $w \in L$, $|w| \geq n$, depending on n — usually blocks of length n : $a^n b^n c^n$.
3. The window uyv has length $\leq n$, so it **can touch at most two neighbouring blocks** — but it can be **anywhere** (unlike the regular lemma). List the cases by which blocks u and v touch.
4. In each case choose i (0 or 2) that breaks membership.
5. All cases lead to contradiction $\Rightarrow$ not CF.

Example 1: $\{a^k b^k c^k\}$

Assume CF, constant n . $w = a^n b^n c^n$. Since $|uyv| \leq n$, uv can't contain both a 's and c 's (the b -block of length n is between them). Case 1: uv has no c . Then pumping $i = 0$ removes at least one a or b but no c : the counts are no longer equal. Case 2: uv has no a : symmetric, $i = 0$ leaves n a 's and fewer b 's or c 's. (Also if u or v contains two different letters, $i = 2$ puts letters out of order.) Contradiction — not CF.

Example 2: $\{ww \mid w \in \{a, b\}^*\}$

$w = a^n b^n a^n b^n$. The window touches at most two adjacent blocks. $i = 0$: if the window is in the first half, the first half loses letters but the second doesn't; the string (if even length) is no longer of the form ww because its middle shifts into the b -blocks mismatching. Case analysis (first half / middle / second half) gives a contradiction each time. Not CF. (Note: $\{ww^R\}$ is CF — mirror, not copy.)

Example 3: $\{a^k \mid k \text{ prime}\}, \{a^{k^2}\}$

Over one letter, CF = regular, so these are not CF either (same proofs as Ch. 6 with $|uv|$ in place of $|y|$).

FCS EXAM Q4 (2022/2025 COVER): THE LANGUAGE PRINTED IS ACTUALLY CONTEXT-FREE

$L = \{w \mid |w|_a > 2|w|_b\} \cup \{w \mid 2|w|_a < |w|_b\}$. Each part is recognised by a PDA that uses its stack as a counter (add 1 per a , subtract 2 per b ; accept if the counter is positive), and CFLs are closed under union. So L is context-free, and no correct proof of the opposite exists. A grammar for it (checked on every string up to length 12):

$S \rightarrow A \mid B, A \rightarrow EaE \mid EaA, B \rightarrow FbF \mid FbB, E \rightarrow aEaEbE \mid aEbEaE \mid bEaEaE \mid \lambda, F \rightarrow bFbFaF \mid bFaFbF \mid aFbFbF \mid \lambda$

If this happens on your exam: don't freeze. Write briefly why the attempt fails, give the PDA/union argument, and say you believe the question contains a mistake — then move on. If a similar question appears with three letters, e.g. $\{w \in \{a, b, c\}^* \mid |w|_a = |w|_b = |w|_c\}$, it is non-CF: intersect with the regular $a^*b^*c^*$ to get $a^n b^n c^n$ (CFL $\cap$ regular is CF, so if L were CF, $a^n b^n c^n$ would be too).

```
$ python3 verify/fcs_q4.py
```

```
[OK] E: |w|_a = 2|w|_b: checked all 8191 strings over {a,b} up to length 12
[OK] |w|_a > 2|w|_b: checked all 8191 strings over {a,b} up to length 12
[OK] 2|w|_a < |w|_b: checked all 8191 strings over {a,b} up to length 12
[OK] the WHOLE exam language L: checked all 8191 strings over {a,b} up to length 12
```

11.1 Closure properties of CFLs

Operation	CFL	DCFL	Why
Union	yes	no	$S \rightarrow S_1 \mid S_2$
Concatenation	yes	no	$S \rightarrow S_1 S_2$
Star	yes	no	$S \rightarrow S_1 S \mid \lambda$
Intersection	no	no	$\{a^n b^n c^m\} \cap \{a^m b^n c^n\} = \{a^n b^n c^n\}$
Complement	no	yes	if CFL were closed under complement, then $\cap$ via De Morgan
$\cap$ with a regular language	yes	yes	product of PDA and DFA (one stack is enough)

Decidable for CFLs: membership (CYK), emptiness, finiteness. **Undecidable:** whether two CFGs are equivalent, whether a CFG is ambiguous, whether $L(G) = \Sigma^*$.

Part II

Computability and complexity
FCS only

12. Turing machines and the language hierarchy FCS

INTUITION: THE SIMPLEST POSSIBLE COMPUTER

A Turing machine is a finite automaton that can **read and write** on an infinite tape and **move both ways**. That is enough to compute anything any computer can compute (Church–Turing thesis). Unlike a DFA it may **never stop**.

DEFINITION (EXAM APPENDIX)

$M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$: states, input alphabet, tape alphabet $\Gamma \supseteq \Sigma \cup \{\square\}$, transition function $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ (partial), start state, blank $\square \notin \Sigma$, final states.

$\delta(q, a) = (p, b, R)$: in state q reading a : write b , move right, go to p . The machine **halts** when δ is undefined. (Linz) it **accepts** w if it halts in a final state. An **instantaneous description** $x q y$ means: tape contents xy , state q , head on the first symbol of y .

Example: accept $\{a^n b^n \mid n \geq 1\}$

Idea: repeatedly cross off the leftmost a (write x) and the leftmost b (write y).

State	Does
q_0	on a : write x , R, go q_1 . On y : all a 's done — go q_3 to check only y 's remain.
q_1	move R over a 's and y 's; on b : write y , L, go q_2
q_2	move L over y 's and a 's; on x : R, go q_0
q_3	move R over y 's; on $\square$: go final q_4 . (Anything else: no move $\Rightarrow$ halt, reject.)

Trace on $aabb$ (instantaneous descriptions, produced by simulating the machine; it was also checked to accept exactly $a^n b^n$ on every string up to length 10):

$q_0 a a b b \vdash x q_1 a b b \vdash x a q_1 b b \vdash x q_2 a y b \vdash q_2 x a y b \vdash x q_0 a y b \vdash x x q_1 y b \vdash x x y q_1 b \vdash x x q_2 y y \vdash x q_2 x y y \vdash x x q_0 y y \vdash x x y q_3 y \vdash x x y y q_3 \square \vdash x x y y q_4 \square$

12.1 Turing machines computing functions — composition (FCS Q6, 6 p)

A TM computes f if started on v it halts in a final state with $f(v)$ on the tape (head at its start).

MODEL ANSWER: BUILD M_{GF} COMPUTING $G(F(V))$ FROM M_F AND M_G

Idea: run M_f ; when it finishes, the tape holds $f(v)$ — exactly the input M_g needs — so continue with M_g . Rename states so the two machines don't clash (assume $Q_f \cap Q_g = \emptyset$).

$M_{gf} = (Q_f \cup Q_g, \Sigma, \Gamma, \delta, q_{0,f}, \square, F_g)$ with

- $\delta(q, a) = \delta_f(q, a)$ for $q \in Q_f$ where δ_f is defined (run M_f);
- $\delta(q, a) = \delta_g(q, a)$ for $q \in Q_g$ (run M_g);
- for every final state $q \in F_f$ and every $a \in \Gamma$ where $\delta_f(q, a)$ is undefined (i.e. where M_f would halt): hand over to M_g . Since M_f halts with the head at the start of $f(v)$, we need a move into $q_{0,g}$ without changing the tape. With only $\{L, R\}$ moves: $\delta(q, a) = (q', a, R)$ and $\delta(q', b) = (q_{0,g}, b, L)$ for all b , with a fresh state q' (step right and back).
- Final states: F_g only (states of F_f are no longer final).

Assumptions to state: both machines use the same tape alphabet (take the union), halt with the head on the first symbol of the output and leave nothing else on the tape.

12.2 Variants — all equally powerful

Variant	Why no more powerful
Stay option (S move)	replace each S by R then L
Semi-infinite tape	fold the two-way tape: two tracks, one for each half
Off-line (read-only input)	copy the input to the work tape
Multitape (k tapes)	one tape with $2k$ tracks: contents + head markers; each step sweeps the tape (quadratic slowdown)
Multidimensional	encode coordinates on a 1-D tape
Nondeterministic	see below (exponential slowdown)

MODEL ANSWER — FCS Q7 (6 P): “WHY NTMS ARE NOT MORE POWERFUL THAN DTMS”

Every DTM is an NTM with one choice, so NTMs are at least as powerful. Conversely, let N be an NTM. The computations of N on input w form a tree: each node is a configuration (ID), its children are the configurations reachable in one move; since δ gives finitely many choices, each node has at most k children. N accepts w iff some node of the tree is an accepting configuration. A deterministic machine D (conveniently with 3 tapes: input, simulation, choice-sequence) explores this tree **breadth-first**: for $\ell = 1, 2, 3, \dots$ it tries every sequence of choices of length ℓ , simulating N on w following those choices. If N has an accepting computation of length ℓ , D finds it at stage ℓ and accepts; otherwise D never accepts. (Breadth-first, not depth-first, since one branch can be infinite.) Multitape DTMs are equivalent to single-tape DTMs, so $L(N)$ is accepted by a DTM. The price is time: up to k^ℓ branches, an exponential slowdown — but the **set of languages** is the same.

12.3 The universal Turing machine

Encode any TM M as a binary string $\langle M \rangle$ (list its transitions). A **universal TM** U takes $\langle M \rangle w$ and simulates M on w . Like an interpreter running a program — it is why one computer can run any program. It also shows the set of TMs is **countable** (they are strings).

12.4 The hierarchy (FCS Q3, 8 p)

MODEL ANSWER — FILL IN 5 CLASSES, EACH WITH A MACHINE AND A LANGUAGE THAT IS *NOT* IN THE SMALLER CLASS

Class	Represented by	Example language (in this class, not in the smaller)
1. Regular	DFA / NFA / regular expression	$\{a^n \mid n \text{ even}\}, a^*b^*$
2. Deterministic CF	DPDA	$\{a^n b^n \mid n \geq 0\}$
3. Context-free	NPDA / CFG	$\{ww^R \mid w \in \{a, b\}^*\}$
4. Recursive (decidable)	TM that always halts (decider)	$\{a^n b^n c^n \mid n \geq 0\}$
5. Recursively enumerable	TM (may loop on non-members)	the halting problem $H = \{\langle M \rangle w \mid M \text{ halts on } w\}$

Also valid: **context-sensitive** (linear bounded automaton / context-sensitive grammar, e.g. $a^n b^n c^n$), placed between CF and recursive. The grading is on **consistency**: class, machine and language must agree.

13. Decidability, countability, the halting problem FCS

13.1 Recursive vs recursively enumerable

DEFINITIONS

- L is **recursively enumerable (RE)** if some TM accepts exactly L : on $w \in L$ it halts and accepts; on $w \notin L$ it rejects **or runs forever**. (A **recognizer**.)
- L is **recursive (decidable)** if some TM accepts L and **halts on every input**. (A **decider**.)
- A **problem** is decidable if the language of its yes-instances is recursive.

KEY FACTS

- Recursive $\subset$ RE (strict). Recursive languages are closed under complement (swap accept/reject of the decider).
- **If L and $\bar{L}$ are both RE, then L is recursive**: run both recognizers in parallel (one step each); one of them must accept.
- So $\bar{H}$ (the complement of the halting problem) is **not** RE.

FCS Q1 TRUE/FALSE TRAPS

- “It can be correct to say that a Turing machine M is decidable.” — **False**. Languages/problems are decidable; a *machine* is a decider (halts always) or not. (Expected answer: false.)
- “The set of suitably encoded Turing machines is RE.” — **True** (even recursive: a TM can check whether a string is a valid encoding).

13.2 Countability and diagonalization

INTUITION

There are only **countably many** TMs (each is a finite string, and strings can be listed by length). But there are **uncountably many** languages (all subsets of Σ^*). So most languages have no TM at all.

CANTOR: 2^{Σ^*} IS UNCOUNTABLE

Suppose the languages could be listed $L_1, L_2, L_3, \dots$ and list the strings $w_1, w_2, w_3, \dots$. Define $D = \{w_i \mid w_i \notin L_i\}$. For each i , D differs from L_i on the string w_i (flip the diagonal). So D is not in the list — contradiction. Hence there are languages that are **not** RE.

13.3 The halting problem

THEOREM

$H = \{\langle M \rangle w \mid M \text{ halts on } w\}$ is RE but **not recursive**.

RE: the universal TM simulates M on w and accepts if it halts.

Not recursive (proof by contradiction): suppose a TM H_D decides it. Build D : on input $\langle M \rangle$, run H_D on $\langle M \rangle \langle M \rangle$; if it says “halts”, loop forever; if it says “doesn’t halt”, halt. Now run D on $\langle D \rangle$: D halts $\Leftrightarrow H_D$ says D doesn’t halt on $\langle D \rangle \Leftrightarrow D$ doesn’t halt. Contradiction. So no such H_D .

13.4 Reductions

RECIPE: PROVING PROBLEM B IS UNDECIDABLE

1. Pick a known undecidable problem A (usually halting).
2. Assume B is decidable by a decider M_B .
3. Show how to use M_B to decide A: given an instance of A, **construct** (by an algorithm that always halts) an instance of B whose answer is the same.
4. Then A would be decidable — contradiction.

Direction matters: reduce the **known hard** problem **to** the new one.

Example: the state-entry problem

“Given M , w , and a state q : does M ever enter q on w ?” Reduce halting: given $\langle M \rangle$, w , build M' : add a new state q_{new} ; wherever M would halt ($\delta(p, a)$ undefined), make M' go to q_{new} . Then M' enters q_{new} on $w \Leftrightarrow M$ halts on w . A decider for state-entry would decide halting. So state-entry is undecidable.

Example: blank-tape halting

Given M , w , build M_w : starting on a blank tape, it first writes w , then runs M . M_w halts on blank $\Leftrightarrow M$ halts on w . So blank-tape halting is undecidable.

Rice’s theorem (general version): every non-trivial property of the *language* of a TM (“is $L(M)$ empty?”, “finite?”, “regular?”) is undecidable.

14. Complexity: P, NP and NP-completeness FCS

INTUITION

Decidability asks “can it be solved at all?”. Complexity asks “can it be solved **fast**?”. **P** = problems we can *solve* quickly. **NP** = problems where a proposed solution can be *checked* quickly (sudoku: hard to solve, easy to check). Whether $P = NP$ is the famous open problem.

14.1 Definitions

DEFINITIONS

- $f(n) = O(g(n))$ if there are c, n_0 with $f(n) \leq c g(n)$ for all $n \geq n_0$.
- A TM runs in time $T(n)$ if it halts within $T(n)$ steps on every input of length n .
- **P** = languages decided by a **deterministic** TM in polynomial time $O(n^k)$.
- **NP** = languages decided by a **nondeterministic** TM in polynomial time (every branch halts in $O(n^k)$, accept if some branch accepts). Equivalently: languages with a polynomial-time **verifier**: $w \in L$ iff there is a **certificate** c with $|c|$ polynomial in $|w|$ such that a deterministic poly-time machine accepts (w, c) .
- $P \subseteq NP$ (a deterministic machine is a special NTM). Simulating an NTM deterministically takes exponential time (Ch. 12), which is why we don't know whether they are equal.
- Poly-time reduction $A \leq_p B$: a poly-time computable f with $w \in A \iff f(w) \in B$.
- B is **NP-hard** if $A \leq_p B$ for every $A \in NP$; **NP-complete** if also $B \in NP$.

COOK-LEVIN

SAT (is a propositional formula satisfiable?) is NP-complete. Then 3SAT, CLIQUE, VERTEX COVER, HAMILTONIAN PATH, SUBSET-SUM... are NP-complete by chains of reductions ($SAT \leq_p 3SAT \leq_p CLIQUE$).

Consequence: if any NP-complete problem is in P, then $P = NP$. So “If $P=NP$ then there is a polynomial algorithm for SAT” is **True** (FCS Q1).

RECIPE: PROVING PROBLEM B IS NP-COMPLETE

1. **B $\in$ NP**: say what the certificate is and why checking it takes polynomial time.
2. **B is NP-hard**: pick a known NP-complete A (SAT, 3SAT, CLIQUE...) and give a reduction f from **A to B**.
3. Show f runs in polynomial time (output size polynomial).
4. Show $w \in A \Rightarrow f(w) \in B$ **and** $f(w) \in B \Rightarrow w \in A$.

14.2 SAT $\rightarrow$ 3SAT $\rightarrow$ CLIQUE, briefly

SAT $\rightarrow$ 3SAT: convert to CNF with new variables (to avoid blow-up), then split long clauses: $(l_1 \vee l_2 \vee l_3 \vee l_4) \rightarrow (l_1 \vee l_2 \vee y) \wedge (\neg y \vee l_3 \vee l_4)$.

3SAT $\rightarrow$ CLIQUE: for a formula with k clauses, make a vertex per literal occurrence; connect two vertices if they are in different clauses and are not contradictory (x vs $\neg x$). The formula is satisfiable iff the graph has a clique of size k (pick one true literal per clause).

14.3 FCS ☆ Q8: $\diamond$ -EQUATION is NP-complete

$\diamond$ is NAND: $0 \diamond 0 = 0 \diamond 1 = 1 \diamond 0 = 1, 1 \diamond 1 = 0$.

MODEL ANSWER

In NP. Certificate: an assignment of 0/1 to the variables (size $\leq$ input). Check: evaluate both sides bottom-up, one operation per $\diamond$ – linear time – and compare.

NP-hard: reduce SAT. Given a formula φ over $\neg, \wedge, \vee$, translate it recursively: $T(x) = x, T(\neg\alpha) = T(\alpha) \diamond 1, T(\alpha \wedge \beta) = (T(\alpha) \diamond T(\beta)) \diamond 1, T(\alpha \vee \beta) = (T(\alpha) \diamond 1) \diamond (T(\beta) \diamond 1)$. These are correct since $x \diamond 1 = \neg x, \neg(a \diamond b) = a \wedge b$, and $(\neg a) \diamond (\neg b) = a \vee b$. Each subformula is used **once**, so $|T(\varphi)| \leq 5|\varphi|$: polynomial time. Output the equation $T(\varphi) = 1$.

For every assignment, $T(\varphi)$ has the same value as φ . So φ is satisfiable $\Leftrightarrow$ some assignment makes $T(\varphi) = 1 \Leftrightarrow$ the equation has a solution. Hence $\text{SAT} \leq_p \diamond\text{-EQUATION}$, which is therefore NP-complete.

\$ checked on the exam example and 3000 random formulas

Example $((x \diamond 1) \diamond y) \diamond (x \diamond z) = (z \diamond 1)$: solutions $(x, y, z) = [(0, 0, 1), (0, 1, 0)]$

3000 random formulas: translation agrees with the formula on every assignment.

Largest size ratio $|e_\varphi| / |\varphi| = 2.73$ (bounded by a constant $\Rightarrow$ polynomial reduction)

14.4 FCS ☆ Q10: NP is closed under $\cup$ and $\cap$

MODEL ANSWER

Definition: NP is the class of languages L decided by a nondeterministic TM in polynomial time (as above).

Let $L_1, L_2 \in \text{NP}$, decided by NTMs N_1, N_2 in times $p_1(n), p_2(n)$.

Union: NTM N on w : nondeterministically choose $i \in \{1, 2\}$, run N_i on w . Some branch accepts $\Leftrightarrow N_1$ or N_2 has an accepting branch $\Leftrightarrow w \in L_1 \cup L_2$. Time $O(p_1(n) + p_2(n))$, polynomial.

Intersection: NTM N on w : run N_1 on w (nondeterministically); if that branch accepts, run N_2 on w (on a fresh copy of w); accept if it accepts. A branch accepts $\Leftrightarrow$ it contains an accepting run of N_1 followed by one of $N_2 \Leftrightarrow w \in L_1 \cap L_2$. Every branch takes at most $p_1(n) + p_2(n) + O(n)$ steps (copying w), polynomial. (Verifier version: the certificate for $\cap$ is the pair (c_1, c_2) ; for $\cup$ it is (i, c_i) .)

Part III

Haskell from zero

PL — 20 points on every exam

15. Haskell basics: types, functions, lists PL

THE MINDSET

In Java you write **instructions**: “set x to 0, loop, add...”. In Haskell you write **definitions**: “the sum of an empty list is 0; the sum of `x:xs` is `x` plus the sum of `xs`”. A function is like a maths function: same input $\Rightarrow$ always the same output, and it changes nothing else (no side effects). There are no loops — recursion does that job. Nothing is ever modified — you build new values.

15.1 Using GHCi

Run `ghci`, load a file with `:l file.hs`, reload with `:r`, ask a type with `:t`. Every `ghci>` line in this guide is real output.

```
ghci> 2 + 3 * 4
14
ghci> 10 `div` 3
3
ghci> 10 `mod` 3
1
ghci> 7 / 2
3.5
ghci> not (3 > 2) || True
True
ghci> "abc" ++ "def"
"abcdef"
ghci> 'h' : "ello"
"hello"
ghci> head [5,6,7]
5
ghci> tail [5,6,7]
[6,7]
ghci> length [5,6,7]
3
ghci> [1..5]
[1,2,3,4,5]
ghci> [x * 2 | x <- [1..5], odd x]
[2,6,10]
```

FUNCTION APPLICATION BINDS TIGHTEST

`f x + 1` means `(f x) + 1`. `f x y` is `f` applied to two arguments — no commas, no parentheses. To pass an expression, wrap it: `double (3 + 4)`, not `double 3 + 4`.

15.2 Types

Type	Values
<code>Int</code> , <code>Integer</code>	whole numbers (fixed size / unlimited)
<code>Double</code>	decimals
<code>Bool</code>	<code>True</code> , <code>False</code>
<code>Char</code>	<code>'a'</code> (single quotes)

String	"abc" — it is just [Char]
[a]	list: all elements the same type
(a, b)	tuple: fixed size, types may differ
a -> b	function from a to b

Lowercase type names like `a` are **type variables**: “any type”. `length :: [a] -> Int` works for lists of anything (parametric polymorphism). A **constraint** like `Eq a =>` means “any type that supports `==`”; `Ord a` supports `<`; `Num a` supports `+`; `Show a` can be printed.

```
ghci> :t 'a'
'a' :: Char
ghci> :t "hi"
"hi" :: String
ghci> :t (True, 'x')
(True, 'x') :: (Bool, Char)
ghci> :t length
length :: Foldable t => t a -> Int
ghci> :t (==)
(==) :: Eq a => a -> a -> Bool
```

Note: modern GHC shows `length :: Foldable t => t a -> Int` because it works on more containers than lists. For lists, read `t a` as `[a]`: `length :: [a] -> Int`. On the exam, the list version is what is expected. The same applies to `sum`, `elem`, `foldr`, `foldl`.

15.3 Defining functions

```
double :: Int -> Int
double x = x * 2

add :: Int -> Int -> Int
add x y = x + y

isTeen :: Int -> Bool
isTeen age = age >= 13 && age <= 19
```

Read `add :: Int -> Int -> Int` as “takes an Int, then an Int, returns an Int”.

```
ghci> double 21
42
ghci> add 3 4
7
ghci> isTeen 15
True
```

Guards: many-way if

```
grade :: Int -> Char
grade points
| points >= 90 = 'A'
| points >= 70 = 'B'
| points >= 50 = 'C'
| otherwise  = 'F'
```

```
ghci> grade 75
'B'
ghci> grade 12
'F'
```

Pattern matching: define by cases on the shape of the input

```
describe :: [Int] -> String
describe []      = "empty"
describe [x]     = "one element: " ++ show x
describe (x:y:_) = "starts with " ++ show x ++ " and " ++ show y
```

Patterns are tried **top to bottom**. `[x]` = exactly one element; `(x:y:_)` = at least two; `_` = “anything, I don’t care”.

```
ghci> describe []
"empty"
ghci> describe [7]
"one element: 7"
ghci> describe [7,8,9]
"starts with 7 and 8"
```

where, let, case

```
bmi :: Double -> Double -> String
bmi weight height
  | v < 18.5 = "under"
  | v < 25.0 = "normal"
  | otherwise = "over"
  where
    v = weight / (height * height)

cylinder :: Double -> Double -> Double
cylinder r h =
  let side = 2 * pi * r * h
      top  = pi * r * r
  in side + 2 * top
```

```
sign :: Int -> String
sign n = case compare n 0 of
  LT -> "negative"
  EQ -> "zero"
  GT -> "positive"
```

```
ghci> bmi 70 1.8
"normal"
ghci> sign (-3)
"negative"
```

Tuples

```
swapPair :: (a, b) -> (b, a)
swapPair (x, y) = (y, x)

minMax :: [Int] -> (Int, Int)
minMax xs = (minimum xs, maximum xs)
```

```
ghci> swapPair (1, "one")
("one",1)
ghci> minMax [3,1,4,1,5]
(1,5)
ghci> fst (1, 'a')
1
ghci> snd (1, 'a')
'a'
```

15.4 Lists: the one data structure you must master

HOW LISTS ARE BUILT

A list is either **empty** `[]` or an element **in front of** a list: `x : xs`. `[1,2,3]` is only shorthand for `1 : (2 : (3 : []))`. That is why every list function has two cases: `[]` and `(x:xs)`.

Function	Does	Cost
<code>x : xs</code>	put in front	$O(1)$
<code>xs ++ ys</code>	join	$O(\text{length } xs)$
<code>head, tail</code>	first / all but first (crash on <code>[]</code>)	$O(1)$
<code>length, last, xs !! n</code>	walk the list	$O(n)$
<code>take n, drop n, splitAt n</code>	first n / without first n / both	$O(n)$
<code>reverse, elem, sum, maximum</code>	as named	$O(n)$
<code>replicate n x</code>	n copies	$O(n)$
<code>zip, unzip</code>	pairs up / splits pairs	$O(n)$
<code>words, unwords, lines</code>	split/join text	$O(n)$

```
ghci> take 2 [1,2,3,4]
[1,2]
ghci> drop 2 [1,2,3,4]
[3,4]
ghci> splitAt 2 "hello"
("he","llo")
ghci> replicate 3 'z'
"zzz"
ghci> zip [1,2,3] "abc"
[(1,'a'),(2,'b'),(3,'c')]
ghci> words "the cat sat"
["the","cat","sat"]
ghci> [1,2,3] !! 0
1
ghci> null []
True
```

16. Recursion and accumulators PL

HOW TO THINK RECURSIVELY (“TRUST THE RECURSION”)

To write `mySum (x:xs)`, **pretend** `mySum xs` **already works**. If `mySum [2,3]` is 5, then `mySum [1,2,3]` is `1 + 5`. You only have to answer two questions: **what’s the answer for the empty list**, and **how do I combine the first element with the answer for the rest**.

RECIPE (USE IT FOR EVERY EXAM FUNCTION)

1. Write the **type signature**.
2. Write the **base case** (`f [] = ...`, or `f 0 = ...`).
3. Write the **recursive case** using `f xs` (smaller input).
4. **Trace** it on a 2–3 element list by hand.

16.1 Pattern 1 — plain recursion

```
mySum :: [Int] -> Int
mySum [] = 0
mySum (x:xs) = x + mySum xs

myProduct :: [Int] -> Int
myProduct [] = 1
myProduct (x:xs) = x * myProduct xs

myElem :: Eq a => a -> [a] -> Bool
myElem _ [] = False
myElem y (x:xs) = x == y || myElem y xs

myTake :: Int -> [a] -> [a]
myTake 0 _ = []
myTake _ [] = []
myTake n (x:xs) = x : myTake (n - 1) xs

myDrop :: Int -> [a] -> [a]
myDrop 0 xs = xs
myDrop _ [] = []
myDrop n (_:xs) = myDrop (n - 1) xs

factorial :: Integer -> Integer
factorial 0 = 1
factorial n = n * factorial (n - 1)
```

```

ghci> mySum [1,2,3,4]
10
ghci> myProduct [1,2,3,4]
24
ghci> myElem 3 [1,2,3]
True
ghci> myTake 2 "hello"
"he"
ghci> myDrop 2 "hello"
"llo"
ghci> factorial 20
2432902008176640000

```

Trace $\text{mySum } [1,2,3] = 1 + \text{mySum } [2,3] = 1 + (2 + \text{mySum } [3]) = 1 + (2 + (3 + \text{mySum } [])) = 1 + (2 + (3 + 0)) = 6$. Notice the additions wait until the end — the computer must remember them on the stack.

Sorting — classic exam material

```

insertSorted :: Int -> [Int] -> [Int]
insertSorted x [] = [x]
insertSorted x (y:ys)
  | x <= y    = x : y : ys
  | otherwise = y : insertSorted x ys

insertionSort :: [Int] -> [Int]
insertionSort [] = []
insertionSort (x:xs) = insertSorted x (insertionSort xs)

quicksort :: [Int] -> [Int]
quicksort [] = []
quicksort (p:xs) = quicksort [x | x <- xs, x < p] ++ [p] ++ quicksort [x | x <- xs, x >= p]

merge :: [Int] -> [Int] -> [Int]
merge [] ys = ys
merge xs [] = xs
merge (x:xs) (y:ys)
  | x <= y    = x : merge xs (y:ys)
  | otherwise = y : merge (x:xs) ys

mergeSort :: [Int] -> [Int]
mergeSort [] = []
mergeSort [x] = [x]
mergeSort xs = merge (mergeSort l) (mergeSort r)
  where (l, r) = splitAt (length xs `div` 2) xs

```

```

ghci> insertionSort [5,2,9,1,5,6]
[1,2,5,5,6,9]
ghci> quicksort [5,2,9,1,5,6]
[1,2,5,5,6,9]
ghci> mergeSort [5,2,9,1,5,6]
[1,2,5,5,6,9]

```

16.2 Pattern 2 — accumulators and tail recursion

INTUITION

Carry the “answer so far” as an extra argument, like a loop variable. When the list is empty, the accumulator **is** the answer. The recursive call is the **last** thing the function does — that is **tail recursion**: nothing waits on the stack, so it runs like a loop in constant stack space.

Template

```
template :: [Int] -> Int
template xs = go xs 0
  where
    go []      acc = acc
    go (y:ys) acc = go ys (acc + y)
```

myRev in O(n) — asked on both 2023 exams (10 p)

```
myRev :: [a] -> [a]
myRev xs = go xs []
  where
    go []      acc = acc
    go (y:ys) acc = go ys (y : acc)
```

```
ghci> myRev "Hello"
"olleH"
ghci> myRev [1,2,3]
[3,2,1]
```

Trace: `go [1,2,3] []` → `go [2,3] [1]` → `go [3] [2,1]` → `go [] [3,2,1]`. One `:` per element = $O(n)$. The accumulator works like a stack of plates: the first element ends up at the bottom.

NOT O(N)

```
slowRev :: [a] -> [a]
slowRev []      = []
slowRev (x:xs) = slowRev xs ++ [x]      -- correct, but  $O(n^2)$ !
```

`++` copies its left list each time: $1 + 2 + \dots + n = O(n^2)$.

Tail-recursive Fibonacci (2017 exam)

```
fibs :: Int -> [Int]
fibs n = go n 0 1
  where
    go 0 _ _ = []
    go k a b = a : go (k - 1) b (a + b)

fibTail :: Int -> Int      -- the n-th Fibonacci number, tail recursive
fibTail n = go n 0 1
  where
    go 0 a _ = a
    go k a b = go (k - 1) b (a + b)
```

```
ghci> fibs 10
[0,1,1,2,3,5,8,13,21,34]
ghci> fibTail 30
832040
```

Naive `fib n = fib (n-1) + fib (n-2)` is correct but takes exponential time (recomputes everything) and is not tail recursive.

Digits and numbers

```
listToInt :: [Int] -> Int
listToInt xs = go xs 0
  where
    go []      acc = acc
    go (d:ds) acc = go ds (acc * 10 + d)
```

```
intToList :: Int -> [Int]
intToList n
  | n < 10    = [n]
  | otherwise = intToList (n `div` 10) ++ [n `mod` 10]
```

```
ghci> listToInt [1,2,3]
123
ghci> intToList 4071
[4,0,7,1]
```

Run-length: compress and decompress (2023 exam, your last exam)

```
compress2 :: String -> [(Char, Int)]
compress2 []      = []
compress2 (c:cs) = go c 1 cs
  where
    go cur n []      = [(cur, n)]
    go cur n (x:xs)
      | x == cur     = go cur (n + 1) xs
      | otherwise    = (cur, n) : go x 1 xs
```

Alternative with `takeWhile`/`dropWhile`

```
compress :: String -> [(Char, Int)]
compress []      = []
compress (c:cs) = (c, 1 + length same) : compress rest
  where
    same = takeWhile (== c) cs
    rest = dropWhile (== c) cs
```

```
decompress :: [(Char, Int)] -> String
decompress []      = []
decompress ((c, n) : ps) = replicate n c ++ decompress ps
```

Without library functions

```
decompress2 :: [(Char, Int)] -> String
decompress2 []      = []
decompress2 ((c, 0) : ps) = decompress2 ps
decompress2 ((c, n) : ps) = c : decompress2 ((c, n - 1) : ps)
```

```
ghci> compress2 "aaaabbcccb"  
[('a',4),('b',2),('c',3),('b',1)]  
ghci> decompress [('a',4),('b',2),('c',3),('b',1)]  
"aaaabbcccb"
```

17. Higher-order functions: map, filter, fold, lambda

PL

FUNCTIONS ARE VALUES

In Haskell a function can be **passed as an argument** and **returned as a result** (“first-class citizens”). A function that takes or returns a function is **higher-order**. `map`, `filter` and `fold` capture the three most common loops once and for all: **transform every element, keep some elements, combine all elements into one value**.

17.1 Lambdas, currying, sections, composition

```
applyTwice :: (a -> a) -> a -> a
applyTwice f x = f (f x)
```

```
ghci> applyTwice (\x -> x * 3) 2
18
ghci> applyTwice (++ "!") "hi"
"hi!!"
ghci> (\x y -> x * y + 1) 3 4
13
```

- **Lambda:** `\x -> x * 3` is a function without a name.
- **Currying:** every function takes one argument at a time. `add3 :: Int -> Int -> Int -> Int` really means `Int -> (Int -> (Int -> Int))`. So `add3 1` is a function waiting for two more (**partial application**).
- **Sections:** `(+ 5)`, `(* 2)`, `(> 3)`, `(`div` 2)` are operators with one side filled in.
- **Composition:** `(f . g) x = f (g x)` — read right to left. `f $ x = f x` (saves parentheses).

```
add3 :: Int -> Int -> Int -> Int
add3 x y z = x + y + z

addFive :: Int -> Int
addFive = (+ 5)
```

```
ghci> :t add3 1
add3 1 :: Int -> Int -> Int
ghci> (add3 1 2) 10
13
ghci> map (add3 1 2) [10, 20]
[13,23]
ghci> addFive 1
6
ghci> filter (> 3) [1,5,2,8]
[5,8]
```

```
countLongWords :: String -> Int
countLongWords = length . filter (\w -> length w > 3) . words
```

```
ghci> countLongWords "the quick brown fox jumps"
3
```

17.2 map and filter — write them yourself

```
map :: (a -> b) -> [a] -> [b]
map f []      = []
map f (x:xs) = f x : map f xs
```

```
filter :: (a -> Bool) -> [a] -> [a]
filter p []      = []
filter p (x:xs)
  | p x      = x : filter p xs
  | otherwise = filter p xs
```

```
myZipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
myZipWith f (x:xs) (y:ys) = f x y : myZipWith f xs ys
myZipWith _ _      = []
```

```
myTakeWhile :: (a -> Bool) -> [a] -> [a]
myTakeWhile _ [] = []
myTakeWhile p (x:xs)
  | p x      = x : myTakeWhile p xs
  | otherwise = []
```

```
ghci> myZipWith (+) [1,2,3] [10,20,30]
[11,22,33]
ghci> myTakeWhile (< 4) [1,2,3,7,1]
[1,2,3]
ghci> dropWhile (< 4) [1,2,3,7,1]
[7,1]
```

17.3 Folds

INTUITION

`foldl f z [x1,x2,x3] = f (f (f z x1) x2) x3` — start with `z`, eat the list from the **left**, accumulator first.

`foldr f z [x1,x2,x3] = f x1 (f x2 (f x3 z))` — replace each `:` by `f` and `[]` by `z`, element first.

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f acc []      = acc
foldl f acc (x:xs) = foldl f (f acc x) xs
```

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f acc []      = acc
foldr f acc (x:xs) = f x (foldr f acc xs)
```

```
ghci> foldl (-) 0 [1,2,3]
-6
ghci> foldr (-) 0 [1,2,3]
2
ghci> foldr (:) [] [1,2,3]
[1,2,3]
ghci> foldl (\acc x -> x : acc) [] [1,2,3]
[3,2,1]
```

`foldl (-) 0 [1,2,3]`

`foldr (-) 0 [1,2,3]`

$$((0-1)-2)-3 = -6$$

$$1-(2-(3-0)) = 2$$

WHICH FOLD?

- Building a list in the same order (map-like): **foldr**.
- Running total / “reading left to right” (sum, count, listToInt, reverse): **foldl**.
- Works on infinite lists (lazily): only **foldr**.

Exam 2023-05-05 Problem 6 (10 p): explain and evaluate

```
foo x y = filter (\w -> x == w) y
bar x y = foldl (\w -> \q -> (if q /= x then w + 1 else w)) 0 y
```

ANSWER

foo x y = filter keeps the elements *w* of *y* for which *x* == *w*: it returns all occurrences of *x* in *y*. **bar x y** = foldl with accumulator *w* starting at 0; for each element *q* it adds 1 if *q* /= *x*: it counts the elements **different** from *x*. The string has 12 characters, 4 of them 'c':

```
ghci> foo 'c' "aabcccaacbbba"
"cccc"
ghci> bar 'c' "aabcccaacbbba"
8
```

17.4 Pipelines — how Haskell programmers really write

```
sumOfSquaresOfEvens :: [Int] -> Int
sumOfSquaresOfEvens xs = sum (map (^ 2) (filter even xs))
```

```
ghci> sumOfSquaresOfEvens [1..10]
220
```

```
ghci> map (\(c, n) -> replicate n c) [('a',2),('b',3)]
["aa","bbb"]
ghci> concat ["ab", "cd"]
"abcd"
ghci> concatMap (\(c, n) -> replicate n c) [('a',2),('b',3)]
"aabbb"
ghci> any even [1,3,5]
False
ghci> all odd [1,3,5]
True
```

18. Data types, type classes, laziness + problem bank

PL

18.1 Algebraic data types

INTUITION

`data` defines a new type by listing its possible **shapes**. Each shape has a **constructor** (capitalised) that may carry values. You take values apart with pattern matching — exactly like lists (which are themselves just `data List a = Nil | Cons a (List a)`).

```
data Shape = Circle Double | Rect Double Double
  deriving (Show, Eq)
```

```
area :: Shape -> Double
area (Circle r) = pi * r * r
area (Rect w h) = w * h
```

```
ghci> area (Circle 1)
3.141592653589793
ghci> area (Rect 3 4)
12.0
ghci> map area [Circle 1, Rect 2 5]
[3.141592653589793,10.0]
```

Maybe — “a value, or nothing”

```
safeHead :: [a] -> Maybe a
safeHead [] = Nothing
safeHead (x:_) = Just x

safeDiv :: Int -> Int -> Maybe Int
safeDiv _ 0 = Nothing
safeDiv a b = Just (a `div` b)

lookupAge :: String -> [(String, Int)] -> Maybe Int
lookupAge _ [] = Nothing
lookupAge k ((name, age) : rest)
  | k == name = Just age
  | otherwise = lookupAge k rest
```

```
ghci> safeHead ([] :: [Int])
Nothing
ghci> safeHead [4,5]
Just 4
ghci> safeDiv 10 0
Nothing
ghci> lookupAge "bo" [("al", 30), ("bo", 25)]
Just 25
```

18.2 Recursive data types and type constructors

The syllabus names “recursive data types” and “type constructors”. `List` and `Tree` below are **type constructors**: they are not types by themselves but build a type from another (`Tree Int`, `Tree String`).

```
data List a = Nil | Cons a (List a)
  deriving Show

lenL :: List a -> Int
lenL Nil      = 0
lenL (Cons _ xs) = 1 + lenL xs
```

```
ghci> lenL (Cons 'a' (Cons 'b' Nil))
2
```

```
data Tree a = Leaf | Node (Tree a) a (Tree a)
  deriving Show

insert :: Ord a => a -> Tree a -> Tree a
insert x Leaf = Node Leaf x Leaf
insert x t@(Node l v r)
  | x < v      = Node (insert x l) v r
  | x > v      = Node l v (insert x r)
  | otherwise = t

member :: Ord a => a -> Tree a -> Bool
member _ Leaf = False
member x (Node l v r)
  | x < v      = member x l
  | x > v      = member x r
  | otherwise = True

inorder :: Tree a -> [a]
inorder Leaf      = []
inorder (Node l v r) = inorder l ++ [v] ++ inorder r

fromList :: Ord a => [a] -> Tree a
fromList = foldr insert Leaf

size :: Tree a -> Int
size Leaf      = 0
size (Node l _ r) = size l + 1 + size r

depth :: Tree a -> Int
depth Leaf      = 0
depth (Node l _ r) = 1 + max (depth l) (depth r)
```

```
ghci> fromList [5,3,8,1]
Node Leaf 1 (Node (Node Leaf 3 (Node Leaf 5 Leaf)) 8 Leaf)
ghci> inorder (fromList [5,3,8,1,4])
[1,3,4,5,8]
ghci> member 4 (fromList [5,3,8,1,4])
True
ghci> size (fromList [5,3,8,1,4])
5
ghci> depth (fromList [5,3,8,1,4])
3
```

Recipe for trees = same as lists: one equation per constructor; recurse into sub-trees.

An expression tree and its evaluator (an interpreter in 6 lines)

```
data Expr = Num Int
          | Add Expr Expr
          | Mul Expr Expr
          | Neg Expr
  deriving Show

eval :: Expr -> Int
eval (Num n)    = n
eval (Add a b)  = eval a + eval b
eval (Mul a b)  = eval a * eval b
eval (Neg e)    = negate (eval e)
```

```
ghci> eval (Add (Num 2) (Mul (Num 3) (Num 4)))
14
```

18.3 Type classes (ad hoc polymorphism)

A type class is a set of types that support certain operations — like a Java interface. `Eq`, `Ord`, `Show`, `Num` are builtin; `deriving (Show, Eq)` generates instances automatically.

```
class Describable a where
  describe :: a -> String

instance Describable Bool where
  describe True  = "yes"
  describe False = "no"

instance Describable Shape where
  describe (Circle _) = "a round shape"
  describe (Rect _ _) = "a box"
```

```
ghci> describe True
"yes"
ghci> describe (Rect 1 2)
"a box"
```

18.4 An ADT in Haskell (information hiding via modules)

```
newtype Stack a = Stack [a]

empty :: Stack a
empty = Stack []

push :: a -> Stack a -> Stack a
push x (Stack xs) = Stack (x : xs)

pop :: Stack a -> Maybe (Stack a)
pop (Stack []) = Nothing
pop (Stack (_:xs)) = Just (Stack xs)

top :: Stack a -> Maybe a
top (Stack []) = Nothing
top (Stack (x:_)) = Just x

isEmpty :: Stack a -> Bool
isEmpty (Stack xs) = null xs
```

In a real module you export `module Stack (Stack, empty, push, pop, top, isEmpty)` where — the type `Stack` without its constructor. Clients can't see or build the list inside: that is **information hiding**.

```
ghci> top (push 3 (push 2 empty))
Just 3
ghci> isEmpty empty
True
```

18.5 Laziness

Haskell evaluates an expression only when its value is needed (**lazy / non-strict** evaluation, call-by-need). So infinite lists are fine as long as you use only a finite part (this is the lecture's `squares` example).

```
naturals :: [Integer]
naturals = [0..]

squares :: [Integer]
squares = [n * n | n <- naturals]

fibsLazy :: [Integer]
fibsLazy = 0 : 1 : zipWith (+) fibsLazy (tail fibsLazy)
```

```
ghci> take 5 squares
[0,1,4,9,16]
ghci> 16 `elem` takeWhile (<= 16) squares
True
ghci> take 10 fibsLazy
[0,1,1,2,3,5,8,13,21,34]
```

18.6 Problem bank (verified answers)

Try each one yourself first. Then compare.

```
countOcc :: Eq a => a -> [a] -> Int
countOcc _ [] = 0
countOcc y (x:xs)
  | x == y    = 1 + countOcc y xs
  | otherwise = countOcc y xs
```

```
myLength :: [a] -> Int
myLength xs = go xs 0
  where
    go []      acc = acc
    go (_:ys) acc = go ys (acc + 1)
```

```
mapF :: (a -> b) -> [a] -> [b]
mapF f xs = foldr (\x acc -> f x : acc) [] xs

filterF :: (a -> Bool) -> [a] -> [a]
filterF p xs = foldr (\x acc -> if p x then x : acc else acc) [] xs
```

```
myLast :: [a] -> a
myLast [x] = x
myLast (_:xs) = myLast xs

myMaximum :: [Int] -> Int
myMaximum [x] = x
myMaximum (x:xs) = max x (myMaximum xs)

elemAt :: [a] -> Int -> a
elemAt (x:_) 0 = x
elemAt (_:xs) n = elemAt xs (n - 1)
```

```
isPal :: Eq a => [a] -> Bool
isPal xs = xs == rev xs []
  where
    rev []      acc = acc
    rev (y:ys) acc = rev ys (y : acc)
```

```
dedup :: Eq a => [a] -> [a]
dedup (x:y:rest)
  | x == y    = dedup (y : rest)
  | otherwise = x : dedup (y : rest)
dedup xs = xs           -- 0 or 1 element left
```

```
myZip :: [a] -> [b] -> [(a, b)]
myZip (x:xs) (y:ys) = (x, y) : myZip xs ys
myZip _ _          = []

myAppend :: [a] -> [a] -> [a]
myAppend []      ys = ys
myAppend (x:xs) ys = x : myAppend xs ys
```

```
ghci> countOcc 'a' "banana"
3
ghci> myLength "abcd"
4
ghci> mapF (*10) [1,2]
[10,20]
ghci> myMaximum [3,9,2]
9
ghci> isPal "racecar"
True
ghci> dedup [1,1,2,3,3,3,1]
[1,2,3,1]
ghci> myAppend "ab" "cd"
"abcd"
```

READING TYPE ERRORS

“Couldn’t match expected type `[a]` with actual type `a`” almost always means you wrote `xs ++ x` (element where a list was needed) — use `xs ++ [x]` or `x : xs`. “Non-exhaustive patterns” means you forgot a case (usually `[]`).

Part IV

Programming-language concepts
PL — the “explain” questions

19. Names, binding, scope, variables and types PL

HOW TO ANSWER ANY “EXPLAIN CONCEPT X” QUESTION

(1) precise definition, (2) how it works / the mechanism, (3) a tiny example, (4) advantages and disadvantages, (5) how it differs from the related concept. Five sentences like that are a full-points answer.

19.1 Paradigms in one table

Paradigm	Program is...	Languages	Key ideas
Imperative	a sequence of commands that change state	C, Pascal, Fortran	variables, assignment, loops
Object-oriented	objects sending messages	Smalltalk, Java, C++, Python	encapsulation, inheritance, dynamic binding
Functional	expressions / function definitions	Lisp, Scheme, ML, Haskell, Erlang	no side effects, recursion, higher-order functions
Logic	facts + rules; you ask queries	Prolog	say <i>what</i> , not <i>how</i> ; the proof engine searches

19.2 Variables and binding

A VARIABLE HAS SIX ATTRIBUTES

name, **address** (where in memory), **value**, **type**, **lifetime** (when it exists in memory), **scope** (where in the program text it can be used).

BINDING AND BINDING TIME

A **binding** is an association between a name and something (a value, a type, an address, an operation). **Binding time** is when it happens: language design (* means multiplication), implementation (size of `int`), compile time (type of a variable in Java), load time (address of a static variable), run time (value of a variable; address of a local). **Static** binding = before run time and doesn't change; **dynamic** = during run time or can change.

Storage kind	Allocated	Example
Static	before execution, whole run	globals, C <code>static</code>
Stack-dynamic	when its declaration is executed (function call), freed at return	local variables
Explicit heap-dynamic	by explicit command	<code>new / malloc</code> , freed by <code>delete</code> or GC
Implicit heap-dynamic	when assigned	Python/JS values

Memory layout (lecture slide): stack (activation records: return address, saved stack pointer, parameters, locals) at the top growing down, free memory, heap growing up, global/static data and constants, program code.

19.3 “Variables” in functional vs imperative languages (2023-05-05 Problem 5, 6 p)

MODEL ANSWER: WHY IS $A = B + 4$ “ASSIGNMENT” IN IMPERATIVE LANGUAGES BUT “NAMING/BINDING A VALUE” IN FUNCTIONAL ONES?

In an **imperative** language a variable is an abstraction of a **memory cell**: $a = b + 4$ evaluates $b + 4$ and **stores** the result in the cell named a , overwriting the previous contents. The value of a can change later ($a = a + 1$ is meaningful), so the meaning of a depends on *when* you look — the program has **state** and assignment is a **side effect**.

In a **pure functional** language there is no mutable state: $a = b + 4$ is a **definition** — it binds the name a to the value of $b + 4$, like in mathematics, and that binding can never change within its scope. $a = a + 1$ is not an update but a (recursive, non-terminating) definition. Because names always stand for the same value, any occurrence of a can be replaced by its value (**referential transparency**), which makes programs easier to reason about, test, parallelise and transform. Repetition is expressed by recursion instead of by changing loop variables. The difference is thus **mutable storage location** vs **immutable name for a value**.

19.4 Scope

DEFINITIONS

- **Scope**: the part of the program text where a name is visible. **Lifetime**: the time during execution when the variable exists. They differ: a C `static local` has local scope but lives for the whole run.
- **Static (lexical) scope**: a free name refers to the declaration in the nearest **enclosing block in the program text**. Decided at compile time. (Almost all languages: C, Java, Python, Haskell.)
- **Dynamic scope**: a free name refers to the most recent binding in the **chain of active calls** at run time. (Early Lisp, bash, Perl’s `local`.)
- **Block structure**: blocks can be nested, and inner declarations **shadow** outer ones with the same name.

The **same program** in a statically scoped language (Python) and a dynamically scoped one (bash):

```
$ python3 scope.py      # static scope
f sees x = 1
```

```
$ bash scope.sh         # dynamic scope
f sees x = 2
```

Python: `f` is defined at top level, so its `x` is the global one (1) — the text decides. Bash: `f` was called from `g`, whose `local x=2` is the most recent binding — the call chain decides. **Static**: readable, checkable at compile time, faster. **Dynamic**: flexible but hard to understand; can't type-check statically; a function's behaviour depends on who called it.

19.5 Types and type systems

Concept	Meaning
Type system	rules that assign types to expressions and define which operations are allowed on which types
Type checking	making sure operands have compatible types; a violation is a type error
Static typing	types known and checked at compile time (Java, C, Haskell). Errors found early, faster code.
Dynamic typing	types belong to values and are checked at run time (Python, JS, Lisp). Flexible, errors later, slower.
Strong typing	all type errors are detected (Haskell, Java nearly); weak : some slip through (C: casts, unions)
Type inference	the compiler deduces types without annotations (Haskell, ML: Hindley–Milner)
Coercion	automatic conversion (<code>1 + 2.5</code> in Java converts 1 to double); weakens error detection
Type equivalence	name equivalence (same declared type name) vs structural (same structure)

The lecture example `def example(a,b): return a + b` works on numbers, strings and lists — dynamic typing plus overloaded `+` (“duck typing”).

20. Control structures and subprograms PL

20.1 Control structures (lecture 10)

An imperative language is mostly `var = expr; .` To be **Turing complete** it needs two more things: **conditions** (simplest: `if`) and **jumps** (simplest and most powerful: `goto`). Structured programming replaces `goto` with readable forms:

- **Selection:** nested `if-then-else` (always use blocks — the dangling-else problem), `elif`; multi-way `switch/case` (with or without `break/fall-through`, default case); **pattern matching** (Haskell, ML).
- **Logical loops:** `while (e) ...` (pre-test), `do ... while (e)` (post-test, runs at least once); control with `break` (leave the loop) and `continue` (skip to the next iteration).
- **Counter loops and iterators:** `for x in fruit` iterates over a collection (list, tree...) instead of an index — simpler and more flexible.
- **Guarded commands** (Dijkstra): `if g1 -> s1 [] g2 -> s2 fi` picks **any** true guard (nondeterministically; error if none). `do ... od` repeats until no guard is true. The lecture's example sorts four variables by swapping any out-of-order neighbours until none is — the order of swaps doesn't matter. Used to reason about correctness and concurrency.

20.2 Subprograms

- Process abstraction: **procedures** do something (change state), **functions** return a value, **methods** are procedures/functions belonging to objects/classes.
- A header defines name, parameters and result type.
- **Design issues** (slide): when/how are parameter types checked? Nested subprograms allowed? Subprograms as parameters? Overloading (same name, different parameter types)? Generic subprograms (one definition for many types)? Side effects allowed? What can be returned, and how many values?
- **Co-routines:** not master/slave — two routines pass control back and forth, each resuming where it left off (several entry points). Used for quasi-concurrency, generators, pipelines.

```
$ python3 coroutine.py
producer: made a
consumer: used a
producer: made b
consumer: used b
producer: made c
consumer: used c
```

Parameters in Python (lecture examples, run in Python 3)

```
$ python3 params.py
20 12
1 ()
1 (2, 3, 4, 5)
20 12 {'myname': 'Halvard'}
```

Positional vs **keyword** parameters, **default** values, **variable number** of arguments (`*args` → tuple, `**kwargs` → dictionary).

Activation records. Each call pushes an activation record (return address, link to caller's record, parameters, local variables) on the run-time stack; it is popped at return. That's why recursion works: each call has its own locals.

20.3 Parameter passing: by value, by reference and the others

THE IDEA IN PLAIN WORDS

By value = you give the function a **photocopy**. It can scribble on the copy; your original is untouched. **By reference** = you give the function **the address of your original**. Whatever it writes, it writes on your original.

20.3.1 Vocabulary first

- **Formal parameter:** the name in the function *definition* (`x` in `void f(int x)`).
- **Actual parameter (argument):** the expression in the *call* (`a+1` in `f(a+1)`).
- **Modes** (semantic models): **in** (data goes into the function), **out** (data comes back to the caller), **in-out** (both).
- **Implementation:** essentially either **copy the value** or **pass an access path (address)**.

20.3.2 The five methods

Method	Mode	How it works	Problems
By value	in	The value of the actual parameter is copied into the formal parameter, which acts as a local variable. Changes inside do not affect the caller.	Copying large data (arrays, structs) costs time and memory.
By result	out	No value is passed in. At return, the formal parameter's final value is copied back to the actual parameter (which must be a variable).	<code>procA(a, a)</code> : two results copied to the same variable — the order of copying decides the value. <code>procA(a[i], i)</code> : is the address of <code>a[i]</code> computed at the call or at the return?
By value-result (copy-in/copy-out)	in-out	Copied in at the call and copied back at the return.	Same aliasing/order problems as by result; copying cost twice. <code>procA(4+4)</code> : an expression cannot receive a result.
By reference	in-out	The address of the actual parameter is passed. The formal parameter is an alias of the caller's variable; every read and write goes to the original.	Aliasing (two names for one variable: <code>f(a, a)</code>); unwanted side effects "open up" the caller's data; indirect access is slightly slower.

By name	in-out	The argument expression is passed <i>unevaluated</i> and re-evaluated every time the parameter is used (like textual substitution; Algol 60).	Hard to understand and slow; basis for lazy evaluation (Haskell uses call-by-need).
----------------	--------	---	---

20.3.3 Verified example

```
#include <iostream>
using namespace std;

void swapByValue(int x, int y)    { int t = x; x = y; y = t; }
void swapByReference(int &x, int &y) { int t = x; x = y; y = t; }

void fRef(int &x, int &y) { x = x + 1; y = y * 2; }           // called as fRef(a, a)

int main() {
    int a = 1, b = 2;
    swapByValue(a, b);
    cout << "after swapByValue(a, b):    a = " << a << ", b = " << b << "\n";
    swapByReference(a, b);
    cout << "after swapByReference(a, b): a = " << a << ", b = " << b << "\n";

    int c = 5;
    fRef(c, c);                                           // x and y are ALIASES of c
    cout << "by reference, f(c, c) with c = 5:    c = " << c << "\n";

    // pass-by-value-result, simulated: copy in, work on copies, copy out (left to right)
    int d = 5;
    int x = d, y = d;           // copy in
    x = x + 1; y = y * 2;       // body works on local copies
    d = x; d = y;               // copy out: x first, then y (the last copy wins)
    cout << "by value-result, f(d, d) with d = 5: d = " << d << " (would be " << x
        << " if copied out in the other order)\n";
}
```

```
$ g++ params.cpp && ./a.out
after swapByValue(a, b):    a = 1, b = 2
after swapByReference(a, b): a = 2, b = 1
by reference, f(c, c) with c = 5:    c = 12
by value-result, f(d, d) with d = 5: d = 10 (would be 6 if copied out in the other order)
```

The last two lines are the aliasing problem in numbers: with the **same** function body and the **same** call `f(c, c)`, pass-by-reference gives 12, pass-by-value-result gives 10 (or 6, depending on copy-back order).

MODEL ANSWER: “EXPLAIN THE DIFFERENCES BETWEEN PASS-BY-VALUE AND PASS-BY-REFERENCE. ADVANTAGES AND DISADVANTAGES?” (6 P)

Pass-by-value: at the call, the *value* of the actual parameter is evaluated and **copied** into the formal parameter, which then behaves like a local variable. Changes to it inside the subprogram do not affect the caller (in-mode). *Advantages:* safe — no side effects on the caller’s variables, easy to reason about; the argument can be any expression ($f(a+1)$); access to the parameter is fast (local). *Disadvantages:* copying large objects costs time and memory; it cannot be used to return results through parameters.

Pass-by-reference: the *address* of the actual parameter is passed, so the formal parameter is an **alias** for the caller’s variable; all reads and writes go directly to the original (in-out mode). *Advantages:* efficient — no copying, only an address is passed, even for large structures; the subprogram can return results through parameters (e.g. `swap(a, b)`). *Disadvantages:* side effects — the caller’s data can be changed unintentionally; **aliasing** (e.g. `f(a, a)` , or an array element and the whole array) makes programs hard to understand and verify; each access goes through an indirection (slightly slower); the argument must be a variable, not an arbitrary expression.

Example: `swap(a, b)` works only with pass-by-reference; with pass-by-value it swaps the copies and `a` , `b` are unchanged. (Java passes everything by value, but for objects the value *is* a reference — so the object can be modified, but the caller’s variable cannot be made to point elsewhere.)

21. Abstraction, ADTs and object-orientation PL

THE IDEA IN PLAIN WORDS

Think of a **medicine capsule**. **Encapsulation** is putting the ingredients *together in one capsule*: the data and the operations that work on it are packed into one unit (a class, a module, an ADT).

Information hiding is making the capsule *opaque*: from outside you can't see or touch what is inside; you can only use it the intended way (swallow it = call its public operations).

They are two different things. You can have a capsule you can see through: a Python class groups data and methods (encapsulation) but anyone can still write `s.data = []` (no real hiding).

21.1 Definitions (these are the lecture's own words, expanded)

ENCAPSULATION

An abstract data type is specified as **one syntactic unit** (class, module, package) that contains the **representation** of the data *together with* the **operations** on it. Other parts of the program can use the type **only through that definition**.

Advantages: better **organisation** (everything about stacks is in one place), code is **easier to read and understand**, **easier to modify and maintain**, and units can be **compiled separately**.

INFORMATION HIDING

The **internal representation** of the unit (its data fields, helper functions, implementation details) is **not accessible from outside** the unit. Clients see only the **interface** (the public operations and their types).

Advantages: **reliability** — code that is not “authorised” cannot read or change the internal data, so the unit can guarantee its own **invariants** (e.g. `top` always points at the top element). And **changeability** — the representation can be replaced (array → linked list) without changing any client code, because no client could depend on it. It also lowers what a programmer must know to use the unit.

How languages provide it: access modifiers `private` / `protected` / `public` (Java, C++, C#); separate definition and implementation modules (Modula-2: the definition module is public, the implementation module is hidden); export lists (Haskell modules exporting a type but not its constructor; ML `abstype`).

21.2 The same stack, showing both ideas (compiles and runs)

```
#include <iostream>

class Stack {           // ENCAPSULATION: data + operations in one unit
private:               // INFORMATION HIDING: invisible from outside
    int data[100];
    int top = -1;
public:                // the interface: the only way in
    void push(int x) { if (top < 99) data[++top] = x; }
    int pop()          { return data[top--]; }
    bool isEmpty()     { return top == -1; }
};

int main() {
    Stack s;
    s.push(1); s.push(2); s.push(3);
    while (!s.isEmpty()) std::cout << s.pop() << " ";
    std::cout << "\n";
#ifdef CHEAT
    s.top = 50;         // try to break the stack from outside
#endif
}
```

```
$ g++ Stack.cpp && ./a.out
3 2 1
```

Now “cheat” by compiling with the line `s.top = 50;` switched on — the compiler refuses. *That* is information hiding: the language enforces it.

```
$ g++ -DCHEAT Stack.cpp
error: 'int Stack::top' is private within this context
```

MODEL ANSWER: “EXPLAIN ENCAPSULATION” (WRITE ALL FOUR PARTS)

(1) Definition. Encapsulation means that a data type’s representation and the operations on it are defined together as one syntactic unit (a class, module or package), so the rest of the program uses the type only through that unit’s definition. **(2) Purpose.** It is a way of achieving data abstraction: the user thinks in terms of “a stack with push/pop”, not in terms of arrays and indices. **(3) Example.** A class `Stack` containing the array and the `top` index together with `push`, `pop`, `isEmpty`. **(4) Advantages.** Better organisation of the code, easier to read, understand and modify, and supports separate compilation of the unit. It is often combined with, but is not the same as, information hiding.

MODEL ANSWER: “EXPLAIN INFORMATION HIDING” (WRITE ALL FIVE PARTS)

(1) Definition. Information hiding means that the internal representation and implementation details of a unit are not visible or accessible outside it; clients can only use the public interface. **(2) Mechanism.** E.g. private fields with public methods in Java/C++, an export list in a Haskell module that exports the type but not its constructors, or Modula-2's separate definition and implementation modules. **(3) Example.** In the stack class, data and top are private; `s.top = 50` from outside is a compile error, so nobody can put the stack into an inconsistent state. **(4) Advantages.** Reliability/integrity (invariants cannot be broken by outside code), the implementation can be changed without affecting clients (low coupling), and users need to know less. **(5) Relation to encapsulation.** Encapsulation *groups* data and operations; information hiding *restricts access* to the grouped internals. Together they give an abstract data type. (Possible disadvantage to mention: some overhead — calls to getters/setters — and sometimes less flexibility, e.g. a subclass or a trusted piece of code may legitimately need internal access; that is why languages add *protected* or “friend”.)

21.3 The neighbours — define these precisely too

Term	Precise definition	Tiny example
Abstraction	Representing something by its essential properties and ignoring details that are irrelevant at the current level. Two kinds: process abstraction (subprograms: you call <code>sort(list)</code> without knowing the algorithm) and data abstraction (abstract data types: you use a stack through push/pop without knowing its representation). Humans need it to manage complexity.	<code>sort(xs);</code> a Stack ADT
Abstract data type (ADT)	A data type whose values can only be created and used through a set of operations, defined by what the operations do (e.g. equations <code>pop(push(s,e)) = s</code> , <code>top(push(s,e)) = e</code>), not how. Implemented with encapsulation + information hiding.	stack: empty, push, pop, top, isEmpty
Inheritance	A class (subclass) is defined as an extension of an existing class (superclass): it inherits its fields and methods and can add new ones or override existing ones. Gives code reuse and an “is-a” hierarchy. Single (Java) or multiple (C++).	<code>class Dog extends Animal</code>
Polymorphism	“Many forms”: one name/interface can be used with values of different types . Subtype polymorphism: a variable of type <code>Animal</code> can hold a <code>Dog</code> , and <code>a.speak()</code> runs the Dog version (via dynamic binding). Parametric : one definition works for all types (generics; Haskell <code>length :: [a] -> Int</code>). Ad hoc : overloading — the same name has different implementations for different types (<code>+</code> on ints and floats; Haskell type classes).	<code>animal.speak();</code> length on any list

Dynamic binding	Which method implementation a call runs is decided at run time from the object's actual class (not from the variable's declared type). Flexible but costs a little speed; static binding decides at compile time (faster).	virtual methods in C++, all methods in Java
------------------------	---	---

WHY YOUR ANSWER WAS "TOO VAGUE"

"Encapsulation is hiding the data" is (a) vague and (b) mixes up the two concepts. Examiners want: a precise definition, the mechanism, an example, the advantages, and the difference from the related concept. The model answers above have exactly that structure — use it for *every* "explain the concept X" question.

21.4 The formal ADT specification (lecture 11)

```

ADT stack is elem + boolean +
operations:  empty : -> stack          push : stack × elem -> stack
            pop  : stack -> stack      top  : stack -> elem          isEmpty : stack -> boolean
equations:  isEmpty(empty) = true      isEmpty(push(s,e)) = false
            pop(push(s,e)) = s          pop(empty) = error
            top(push(s,e)) = e          top(empty) = error

```

The equations say **what** the operations do, with no word about arrays or lists — that is data abstraction. A program must also contain implementation details; the ADT idea combines the high-level description with an implementation hidden inside a unit. The lecture shows the same stack in Modula-2 (definition module = interface, implementation module = hidden), Java (`private` fields), ML (`abstype ... with ... end`) and Python (class, but nothing truly hidden).

21.5 Object-oriented programming (lecture 11)

- **Core ideas:** data at the centre, encapsulation, inheritance, dynamic binding, modelling. Objects send and answer **messages**; classes vs instances; class methods (static) vs instance methods.
- **History:** Simula 67 (first classes), Smalltalk (1972, pure OO), C++ and Java (1980s–90s).
- **Language support:** pure OO (Smalltalk, Ruby: everything is an object, simple and elegant but slower); mostly OO (Java: everything except primitive types, which have wrapper classes; objects on the heap; single inheritance + interfaces; C#, Eiffel); OO added to an existing language (C++: mixed type system, multiple inheritance, access control; Objective-C, Ada 95, CLOS).
- **OO vs ADT:** ADTs are hard to **reuse** with small changes; inheritance lets a subclass reuse and modify. But access rules like `protected` let subclasses see internals — this **breaks strict information hiding**. Overriding can cause **shadowing**.
- **Inheritance: single or multiple?** Multiple is more expressive but complex (the **diamond problem**: two parents inherit the same method from a grandparent — which version?). Java: single inheritance + multiple interfaces.
- **Dynamic vs static binding:** dynamic = flexible (polymorphic calls), costs a lookup at run time; static = faster. C++ lets you choose (`virtual`); Java uses dynamic by default.
- **Type checking of overridden methods:** dynamic checking costs time; static checking requires the overriding method to have compatible parameters/return type — cheaper and safer.
- **Where are objects allocated?** Stack (fast, freed automatically, but lifetime tied to the block; C++ allows it) vs heap (flexible lifetime, needs GC or `delete` ; Java always).

- **Nested classes:** a class inside a class — hides a helper class, can access the outer class's members; but more complex.
- **Getters and setters** keep fields private while allowing controlled access; Python's `@property` makes them look like plain attributes.

22. Interpreters: lexer, parser, executor PL

THE IDEA IN PLAIN WORDS

Your program is, to the computer, just a long **string of characters**: `x = 2 + 3 * 4;`. To run it, the language implementation works like reading a sentence in a foreign language:

1. **Lexer (scanner)** — find the *words*. Group characters into meaningful chunks and label them: `x` is an identifier, `=` is an assignment sign, `2` is a number... Throw away spaces and comments.
2. **Parser** — find the *grammar/structure* of the sentence. Check that the words are in a legal order and build a **tree** that shows what belongs to what: the `3 * 4` is done first and then added to `2`.
3. **Executor (interpreter)** — *do what the sentence says*. Walk the tree, compute the values, store `14` in `x`.



The pipeline (Crafting Interpreters calls it “the mountain”)

22.1 Lexer (scanner)

Input	the source code as a string / stream of characters
Output	a list (sequence) of tokens
Task	Group characters into lexemes (the smallest meaningful pieces: <code>while</code> , <code>x1</code> , <code>3.14</code> , <code>>=</code>) and classify each one as a token . A token typically stores: its type (IDENTIFIER, NUMBER, PLUS, keyword WHILE...), the lexeme text, the literal value (e.g. the number 3.14) and the line number (for error messages). It discards whitespace and comments, recognises keywords, and reports lexical errors (an illegal character like <code>#</code> , an unterminated string).
Theory	Tokens are described by regular expressions (identifier = <code>[a-zA-Z_][a-zA-Z0-9_]*</code>), so the lexer is a finite automaton . It follows the “longest match” rule: <code>>=</code> is one token, not <code>></code> then <code>=</code> ; <code>whilex</code> is an identifier, not the keyword <code>while</code> .
Data structures	a character buffer/string with position pointers (start of the current lexeme, current character); a hash table of keywords (reserved words); the output list/array of tokens ; sometimes it inserts identifiers in a symbol table .

22.2 Parser

Input	the list of tokens from the lexer
Output	a syntax tree — usually an abstract syntax tree (AST) — or syntax errors
Task	Check that the token sequence follows the grammar of the language (the syntax), and build a tree that shows the structure of the program: which operands belong to which operator, which statements are inside which loop/block/function. Precedence and associativity are decided here : from <code>2 + 3 * 4</code> the parser builds <code>Add(2, Mul(3, 4))</code> . It reports syntax errors with a position (“expected ‘)’ at line 7”) and tries to recover (skip to the next statement) to report more than one error.

Theory	The syntax is described by a context-free grammar (BNF), because programs have nesting (parentheses, blocks inside blocks) that regular expressions cannot describe. A parser is in effect a pushdown automaton : it needs a stack. Common techniques: recursive descent (top-down: one function per grammar rule; the call stack is the stack), LL, and LR/LALR (bottom-up, e.g. generated by yacc/bison). An ambiguous grammar is a problem for the parser, since one program would get two trees (two meanings).
Data structures	the token list with an index to the current token (lookahead); a stack (explicit in LR parsers, or the recursion call stack in recursive descent); the output tree (nodes = operators, statements; leaves = literals, identifiers); often a parse table (LL/LR).

22.3 Between parser and executor: static (semantic) analysis

Things the grammar cannot check: Is this variable declared? Which declaration does this name refer to (scope resolution)? Do the types fit (in statically typed languages)? This pass walks the tree and uses a **symbol table** (hash maps, one per scope, stacked/chained for nested scopes). It may annotate the tree with the results.

22.4 Executor (the interpreting part)

Input	the (annotated) syntax tree
Output	the program's behaviour: values, output, side effects; or runtime errors
Task	Walks the tree recursively: to evaluate an <code>Add</code> node, evaluate both children then add (post-order). Executes statements in order, handles control flow (if, loops, calls, return, exceptions), looks up and updates variables, calls functions, reports runtime errors (division by zero, type errors in dynamically typed languages, undefined variable).
Data structures	the AST ; an environment = mapping from names to values, implemented as a hash table , with one table per scope linked to its enclosing scope (a chain of hash tables); a call stack of activation records (parameters, local variables, return address) for function calls; a heap for objects, closures, strings. A bytecode interpreter instead uses an array of instructions and a value stack .

INTERPRETER VS COMPILER (OFTEN ASKED TOGETHER)

A **compiler** translates the whole program to another language (e.g. machine code) *before* it runs; the lexer and parser are the same, but the last stage *generates code* instead of executing. An **interpreter** executes the program directly from its tree (or bytecode). Hybrid: compile to bytecode, then interpret it in a virtual machine (Java, Python), possibly with a JIT compiler. Interpreter: easier, portable, good error messages, slower. Compiler: faster execution, errors found before running, less flexible.

22.5 See it: a complete tiny interpreter (Haskell, runs)

For arithmetic like `2 + 3 * 4`. Look at what each stage produces.

Stage 1 — the lexer

```

data Token = TNum Int | TPlus | TMinus | TTimes | TLParen | TRParen
  deriving Show

-- LEXER: characters -> tokens
lexer :: String -> [Token]
lexer [] = []
lexer (c:cs)
  | isSpace c = lexer cs -- skip whitespace
  | isDigit c = TNum (read (c : takeWhile isDigit cs))
    : lexer (dropWhile isDigit cs)
  | c == '+' = TPlus : lexer cs
  | c == '-' = TMinus : lexer cs
  | c == '*' = TTimes : lexer cs
  | c == '(' = TLParen : lexer cs
  | c == ')' = TRParen : lexer cs
  | otherwise = error ("lexical error: unexpected " ++ [c])

```

```

ghci> lexer "2 + 3 * 4"
[TNum 2,TPlus,TNum 3,TTimes,TNum 4]

```

The syntax-tree type

```

data Expr = Num Int | Add Expr Expr | Sub Expr Expr | Mul Expr Expr
  deriving Show

```

Stage 2 — a recursive-descent parser: one function per grammar rule

```

-- PARSE: tokens -> syntax tree. Grammar (one function per rule):
-- expr -> term { ("+" | "-") term } lowest precedence, left-associative
-- term -> factor { "*" factor }
-- factor -> NUMBER | "(" expr ")" highest precedence
parse :: [Token] -> Expr
parse ts = case expr ts of
  (e, []) -> e
  (_, rest) -> error ("syntax error at " ++ show rest)

expr :: [Token] -> (Expr, [Token])
expr ts = let (t, rest) = term ts in loop t rest
  where
    loop acc (TPlus : r) = let (t, r') = term r in loop (Add acc t) r'
    loop acc (TMinus : r) = let (t, r') = term r in loop (Sub acc t) r'
    loop acc r = (acc, r)

term :: [Token] -> (Expr, [Token])
term ts = let (f, rest) = factor ts in loop f rest
  where
    loop acc (TTimes : r) = let (f, r') = factor r in loop (Mul acc f) r'
    loop acc r = (acc, r)

factor :: [Token] -> (Expr, [Token])
factor (TNum n : r) = (Num n, r)
factor (TLParen : r) = case expr r of
  (e, TRParen : r') -> (e, r')
  _ -> error "syntax error: expected )"
factor ts = error ("syntax error at " ++ show ts)

```

```

ghci> parse (lexer "2 + 3 * 4")
Add (Num 2) (Mul (Num 3) (Num 4))
ghci> parse (lexer "10 - 4 - 3")
Sub (Sub (Num 10) (Num 4)) (Num 3)

```

Look at the trees: $*$ ended up **deeper** than $+$ (higher precedence — done first), and $10 - 4 - 3$ became $(10 - 4) - 3$ (left associative). This is exactly what the grammar levels $\text{expr} \rightarrow \text{term} \rightarrow \text{factor}$ produce.

Stage 3 — the executor walks the tree

```
-- EXECUTOR: walk the tree
eval :: Expr -> Int
eval (Num n)    = n
eval (Add a b)  = eval a + eval b
eval (Sub a b)  = eval a - eval b
eval (Mul a b)  = eval a * eval b

run :: String -> Int
run = eval . parse . lexer
```

```
ghci> run "2 + 3 * 4"
14
ghci> run "10 - 4 - 3"
3
ghci> run "(2 + 3) * 4"
20
```

22.6 Precedence and associativity (asked on both 2023 exams, 6 p)

MODEL ANSWER: "BRIEFLY DESCRIBE PRECEDENCE AND ASSOCIATIVITY. HOW ARE THEY RELATED IN THE CONTEXT OF PROGRAMMING LANGUAGES?"

Precedence decides which of two **different** operators binds tighter when there are no parentheses. $*$ has higher precedence than $+$, so $2 + 3 * 4$ means $2 + (3 * 4) = 14$, not $(2 + 3) * 4 = 20$.

Associativity decides how operators of the **same** precedence level are grouped. $-$ is **left-associative**: $10 - 4 - 3 = (10 - 4) - 3 = 3$, not $10 - (4 - 3) = 9$. Exponentiation and assignment are usually **right-associative**: $2 ^ 3 ^ 2 = 2 ^ (3 ^ 2) = 512$; $a = b = 0 = a = (b = 0)$. Some operators are non-associative ($a < b < c$ is an error in some languages).

Relation: both are rules for resolving the **ambiguity** of expressions written without parentheses, i.e. for deciding the shape of the syntax tree. Precedence is applied first (between levels); associativity is only needed when operators have **the same** precedence. In a grammar they are encoded structurally: one nonterminal per precedence level ($\text{expr} \rightarrow \text{expr} + \text{term} \mid \text{term}$, $\text{term} \rightarrow \text{term} * \text{factor} \mid \text{factor}$, $\text{factor} \rightarrow \text{num} \mid (\text{expr})$) — lower levels are deeper in the tree, so they bind tighter; **left recursion** ($\text{expr} \rightarrow \text{expr} + \text{term}$) gives left associativity, **right recursion** ($\text{pow} \rightarrow \text{base} ^ \text{pow}$) gives right associativity. They matter because they change the result ($10-4-3$) and, with side effects, behaviour. Parentheses always override both.

MODEL ANSWER: “DESCRIBE AS MUCH AS THE SPACE PERMITS THE ROLE OF A PARSER IN A PROGRAM”

The parser is the second phase of an interpreter/compiler, after the lexer. Its input is the **sequence of tokens** produced by the lexer (e.g. `ID(x) ASSIGN NUM(2) PLUS NUM(3) STAR NUM(4) .`). Its job is **syntax analysis**: it checks that the token sequence can be derived from the language's **context-free grammar**, and it builds a **syntax tree** (usually an abstract syntax tree) that represents the hierarchical structure of the program — which statements are inside which blocks, and which operands belong to which operators. When building the tree the parser applies the language's **precedence and associativity** rules, so `2+3*4` becomes `+(2, *(3,4))`. If the tokens do not fit the grammar it reports a **syntax error** with its location and tries to recover to find further errors. The tree is the input to the later phases: semantic analysis (scopes, type checking, with a symbol table) and then execution (interpreter) or code generation (compiler). A common technique is **recursive descent**, where each grammar rule becomes a function and the recursion stack plays the role of the pushdown automaton's stack; other parsers are table-driven (LL, LR). Data structures used: the token list with a current position, a stack (explicit or the call stack), and the tree it outputs.

MODEL ANSWER: “AN INTERPRETER CONSISTS OF A SCANNER/LEXER, A PARSER AND AN EXECUTING PART. WHAT ARE THEIR TASKS AND WHAT DATA STRUCTURES DO THEY USE?” (8 P)

Lexer: reads the source text character by character and groups characters into lexemes, which it classifies as tokens (type, lexeme, value, line). Skips whitespace and comments, distinguishes keywords from identifiers, reports lexical errors. Works like a finite automaton defined by regular expressions. *Data structures:* input string/buffer with position pointers, keyword hash table, output list of tokens.

Parser: takes the token list and checks it against the context-free grammar of the language; builds a syntax tree (AST) that captures structure, precedence and associativity; reports syntax errors. Recursive descent or table-driven (LL/LR). *Data structures:* token list + index, a stack (call stack in recursive descent), the tree (and possibly a symbol table).

Executor: traverses the tree and carries out the computation: evaluates expressions recursively, executes statements, handles control flow and function calls, looks up/updates variables, reports runtime errors. *Data structures:* the AST, environments (hash tables mapping names to values, chained for nested scopes), a call stack of activation records (parameters, locals, return address), and a heap for dynamically allocated data.

23. Concurrency, exceptions and events PL

23.1 Concurrency

INTUITION

Several things happen at the same time. **Truly concurrent** = several physical processors. **Logically concurrent** = one processor switching between tasks (simulating several). Levels: machine instructions, expressions/statements, subprograms, whole programs. Our focus: many different things happening at once and **communicating** (not squeezing out performance, HPC).

Why: reality is parallel (modelling); better **responsiveness** (a GUI thread keeps reacting while work goes on); using several cores/processors/machines.

The problem: concurrent tasks must share data and resources. If one thread changes data another is using, the result depends on timing — a **race condition**. Example: two threads do `count = count + 1` (read, add, write); if both read 5 before either writes, both write 6 and one increment is lost. So we need **synchronisation: mutual exclusion** (only one at a time in a **critical section**) and **cooperation** (wait until another task has produced something).

Mechanism	How it works	Pros / cons
Semaphore	a counter with two atomic operations. wait: if counter > 0, decrement and continue, else block. signal: if no one waits, increment; otherwise wake one blocked process.	simple, minimal code; but the programmer is responsible for everything (forget a signal ⇒ deadlock; forget a wait ⇒ race)
Monitor	a language construct: shared data + the operations on it; only one thread can be executing inside at a time; condition wait/signal inside. Like Java <code>synchronized</code> .	safer, mutual exclusion automatic; may lock too much (less parallelism)
Message passing	tasks send messages (letters, mailbox); synchronous (rendezvous, Ada) or asynchronous	easy to understand, no shared memory, works across machines; costs speed and resources
Transactional memory	a block of instructions is atomic like a database transaction; if it can't complete, earlier steps are rolled back	simple for the programmer; overhead, not everywhere available

Deadlock: tasks wait for each other forever (A holds X, wants Y; B holds Y, wants X). **Starvation:** a task never gets its turn.

23.2 Exception handling

Kinds of error: truly unexpected (memory/disk failure), unexpected but possible (overflow, division by zero), expected (end of file). Hardware or software detects it and raises an **exception**: try to recover, or terminate.

You *could* check return values and flags everywhere, but the code becomes messy. Modern languages use **try-catch(-finally)**: clearer code, you think explicitly about which errors can occur, and you can

handle an error at the level that knows what to do — an exception not caught locally **propagates** up the call chain to the nearest matching handler.

Design questions (slide): how is an exception raised; can it carry parameters/information; can it be handled at different levels and how does propagation work; where should errors be handled; how is the right handler chosen among several; is there `finally` (cleanup that always runs)?

23.3 Events

An **event** is not an error but a way of programming: something happens (a click, a key, a timer) and a registered **handler** is called. Events resemble exceptions (both come from hardware signals and interrupt vectors) and messages between threads. Java's listener model is typical.

24. Functional and logic languages PL

24.1 Functional languages

Family: LISP → Scheme, ML, Haskell, Erlang. Defining feature: **no side effects** — a function's result depends only on its arguments. The lecture's `member` in two languages:

```
; Scheme
(DEFINE (member a lst)
  (COND ((NULL? lst) '())
        ((EQ? a (CAR lst)) #T)
        (ELSE (member a (CDR lst)))))

(* ML *)
fun member a []      = false
  | member a (b::l) = a = b orelse member a l
```

In Haskell: `member a [] = False`; `member a (b:l) = a == b || member a l`. Haskell is **purely** functional, handles I/O in a controlled way, and has **non-strict (lazy) semantics**: `squares = [n*n | n <- [0..]]` is an infinite list that is only computed as far as needed.

ADVANTAGES (SLIDE)

Programs are easier to **prove correct** and to **transform** (referential transparency); functions are **first-class citizens**; simpler syntax and semantics; natural for many problems; easy to parallelise (no shared state). Disadvantages: less efficient on conventional hardware, I/O and state are awkward, unfamiliar. Many imperative languages have adopted functional features: lambdas / functions as parameters in Java, JavaScript, Python, Ruby, C#.

24.2 Logic programming (Prolog)

INTUITION

You don't tell the computer *how* to solve the problem. You state **facts** and **rules** about the problem domain, then ask whether a **goal** follows. A built-in **proof engine** searches for an answer (by unification and backtracking). Example: "a sorted list is a permutation of the old list in which every element is $\leq$ the next" — that's the whole sort program.

Queries (run in SWI-Prolog): "What does charlie study?", "Who are kirke's students?", "Who teaches jack?"

```
$ ?- studies(charlie, What).    ?- professor(kirke, Students).    ?- professor(Who, jack).
csc135
charlie
olivia
collins
```

To answer `professor(kirke, S) : find teaches(kirke, C) → C = csc135 ; then studies(S, csc135) → S = charlie`, and on backtracking `S = olivia`. Matching patterns with variables is **unification**; trying the next alternative when one fails is **backtracking**.

Limitations (slide): efficiency (blind search can be very slow); the programmer must still think about the search order (order of rules matters, left recursion can loop forever); answers are only true/fail; **negation** is “not provable” (closed-world assumption), not logical negation. The **cut** ! partly controls the search (stop backtracking). Applications: databases, expert systems, AI, natural language.

Part V

Real exams solved, practice exams, cheat sheets

25. The FCS exam (5DV037), every question answered

FCS

Do it yourself first with a 4-hour timer, then compare.

Q1 (6 p) True/false — -0.5 for each wrong answer

Statement	Answer	Reason
Starts with a / ends with c / odd number of b's — regular	True	union of three regular languages (Ch. 3 regex)
L CF non-regular, L' regular $\Rightarrow L \cup L'$ not regular	False	$L' = \Sigma^*$
L CF non-regular, L' regular $\Rightarrow L \cap L'$ not regular	False	$L' = \emptyset$
$P = NP \Rightarrow$ poly-time algorithm for SAT	True	$SAT \in NP = P$
It can be correct to say a TM M is decidable	False	languages/problems are decidable, not machines
Set of encoded TMs is RE	True	even recursive: a TM checks the encoding

Q2 (5 p) closure: model answer in Ch. 5.1. **Q3** (8 p) hierarchy table: Ch. 12.4.

Q4 (8 p): as printed, the language is context-free, so it can't be proved non-CF — see Ch. 11 for what to write, and the proof technique for languages that really are non-CF ($a^n b^n c^n$).

Q5 (8 p) CNF: **Yes**, it is in CNF. CYK: `abccab` $\in L(G)$. Full table in Ch. 10, Part B.

Q6 (6 p) M_{gf} : Ch. 12.1. **Q7** (6 p) NTM vs DTM: Ch. 12.2.

Q8 ☆ ◇-EQUATION: Ch. 14.3. **Q9** ☆ infinite union: **not** necessarily regular, Ch. 5.4. **Q10** ☆ NP closure: Ch. 14.4.

25.1 Practice exam (new questions)

- (6 p) True/false: (a) Every regular language is context-free. (b) The complement of a CFL is CF. (c) If L is RE and $\bar{L}$ is RE then L is recursive. (d) $\{a^n b^n\} \cap a^* b^*$ is regular. (e) NFAs accept more languages than DFAs. (f) If $A \leq_P B$ and $B \in P$ then $A \in P$.
- (6 p) Give a DFA for strings over $\{a,b\}$ that contain ab and have even length.
- (6 p) Convert the NFA $p_0 \xrightarrow{a,b} p_0$, $p_0 \xrightarrow{b} p_1$, $p_1 \xrightarrow{a} p_2$ (final p_2) to a DFA.
- (8 p) Prove $\{a^n b^m \mid n > m\}$ is not regular.
- (8 p) Convert $S \rightarrow aSb \mid SS \mid \lambda$ to CNF and run CYK on `aabb`.
- (8 p) Prove $\{a^n b^n c^n\}$ is not context-free.
- (6 p) Show that the problem “does TM M ever write the symbol 1 on input w ?” is undecidable.

SOLUTIONS

1. (a) T (b) F (c) T (d) F — it equals $\{a^n b^n\}$ (e) F (f) T.
2. Product of “seen ab” (3 states: none / last was a / seen) and parity (2 states): 6 states, final = (seen, even).
3. Language: ends in ba . DFA states $\{p_0\}, \{p_0, p_1\}, \{p_0, p_2\}$: $\{p_0\} \xrightarrow{a} \{p_0\}, \{p_0\} \xrightarrow{b} \{p_0, p_1\}, \{p_0, p_1\} \xrightarrow{a} \{p_0, p_2\}, \{p_0, p_1\} \xrightarrow{b} \{p_0, p_1\}, \{p_0, p_2\} \xrightarrow{a} \{p_0\}, \{p_0, p_2\} \xrightarrow{b} \{p_0, p_1\}$; final $\{p_0, p_2\}$.
4. $w = a^{n+1}b^n; y = a^j; i = 0$ gives $a^{n+1-j}b^n$ with $n + 1 - j \leq n$: not in L.
5. Nullable S. Remove λ : $S \rightarrow aSb \mid ab \mid SS$ (and $S \rightarrow S$, dropped). CNF: $S \rightarrow AX \mid AB \mid SS, X \rightarrow SB, A \rightarrow a, B \rightarrow b$. CYK on aabb: row1 A,A,B,B; row2 $\emptyset, S, \emptyset$; row3 $\emptyset$ (a|abb: $A \cdot \emptyset$; aa|bb: $\emptyset$), X (abb = ab|b: $S \cdot B \rightarrow X$); row4: a|abb: $A \cdot X \rightarrow S$. $S \in \text{top} \Rightarrow$ accepted.
6. Ch. 11 Example 1.
7. Reduce halting: given M, w, build M' that simulates M using a tape alphabet without 1 (rename 1 to a new symbol), and when M would halt, writes 1. M' writes 1 on w $\Leftrightarrow$ M halts on w.

26. The PL exams (5DV086), every question answered

PL

26.1 Tentamen 2023-03-16

#	Question (translated)	Answer
1 (10)	Which strings can G generate?	None of them. $L(G) = \{a^{k+1}b^k\} \cup \{cb\}$. Ch. 7.3
2 (10)	CFG for “equal a and b, every prefix $\geq$ as many a as b”	$S \rightarrow aSbS \mid \varepsilon$. Ch. 7.3
3 (6)	Precedence and associativity; how related?	Model answer Ch. 8
4 (8)	Parts of an interpreter, tasks, data structures	Model answer Ch. 22
5 (6)	Pass-by-reference vs pass-by-value, pros/cons	Model answer Ch. 20.3
6 (10)	Write <code>compress</code>	Ch. 16 (both versions)
7 (10)	Write <code>myRev</code> , $O(n)$, no library functions	Ch. 16

26.2 Tentamen 2023-05-05

#	Question	Answer
1 (10)	CFG for $\{a^n b^m \mid n \neq 2m\}$	Ch. 7.3 (both readings — say which one you answer)
2–4, 7	same as above	
5 (6)	Why is <code>a = b + 4</code> assignment vs naming a value?	Model answer Ch. 19.3
6 (10)	What do <code>foo</code> , <code>bar</code> do; evaluate them	<code>"cccc"</code> and 8. Ch. 17.3

26.3 Your last exam (what you told me)

Topic	Where
Convert to Chomsky normal form	Ch. 10 Part A
CYK: show table, explain	Ch. 10 Part B
Encapsulation / information hiding	Ch. 21
Role of the parser	Ch. 22
Write <code>foldl</code> ; list to int	Ch. 17.3, Ch. 16
Call by value vs reference	Ch. 20.3
decompress	Ch. 16

Pattern: the PL exam = **grammars (CFG design / CNF / CYK) + 3 theory questions + 2–3 Haskell functions.**

26.4 Practice exam (60 p, 36 to pass) — 4 hours, no aids

1. (10 p) Construct a CFG for $\{a^n b^m c^k \mid k = n + m\}$. Then show a leftmost derivation of $abcc$.
2. (10 p) Convert $S \rightarrow aSb \mid bA \mid \lambda$, $A \rightarrow aA \mid a$ to Chomsky normal form.
3. (6 p) Explain static vs dynamic scoping with an example where they give different results.
4. (8 p) What is polymorphism? Describe three kinds with examples.
5. (6 p) What is the difference between a compiler and an interpreter? What does the lexer produce?
6. (10 p) Write `countRuns :: String -> Int` that counts the number of maximal runs of equal characters (`countRuns "aaabbca" = 4`), and `myFoldr`.
7. (10 p) Write `merge :: [Int] -> [Int] -> [Int]` merging two sorted lists, and use it to write `merge sort`.

SOLUTIONS

1. $S \rightarrow aSc \mid B$, $B \rightarrow bBc \mid \lambda$ (R5). $S \Rightarrow aSc \Rightarrow aBc \Rightarrow abBcc \Rightarrow abcc$.
2. Nullable: S. Remove λ : $S \rightarrow aSb \mid ab \mid bA$, $A \rightarrow aA \mid a$. No units. CNF: $S \rightarrow UX \mid UT \mid TA$, $X \rightarrow ST$, $A \rightarrow UA \mid a$, $U \rightarrow a$, $T \rightarrow b$.
3. Ch. 19.4 (the Python/bash example). 4. Ch. 21.3. 5. Ch. 22.
6. `countRuns = length . compress` (using `compress` from Ch. 16), or directly: `countRuns [] = 0`;
`countRuns (c:cs) = 1 + countRuns (dropWhile (== c) cs)`. `myFoldr` = `foldr` from Ch. 17.
7. Ch. 16 (`merge`, `mergeSort`).

Appendix A. Cheat sheets

AUTOMATA & REGULAR

- DFA 5-tuple; δ total. NFA: $\delta \rightarrow$ sets, λ -moves. NFA $\rightarrow$ DFA: sets of states + λ -closure; final if contains a final.
- Regex: star > concat > union (+).
- Closed: $\cup$ $\cap$ complement concat star reverse difference. Complement needs a complete DFA.
- Pumping: $w = xyz$, $|xy| \leq n$, $|y| \geq 1$. Pick w from n ; all splits; pump 0 or 2.
- Minimize: mark final/non-final pairs; repeat; merge unmarked.

CONTEXT-FREE

- aSb matches pairs. $\neq = < \cup >$. "or" = $S \rightarrow S_1 \mid S_2$.
- Balanced: $S \rightarrow aSbS \mid \lambda$. Equal count: $S \rightarrow aSbS \mid bSaS \mid \lambda$.
- CNF: $\lambda \rightarrow$ units $\rightarrow$ useless $\rightarrow$ terminals $\rightarrow$ split.
- CYK: V_{ij} from $A \rightarrow a$; V_{ij} over all cuts k with $A \rightarrow BC$; $S \in V_{1n} = \text{member}$.
- CF pumping: $xuyvz$, $|uyv| \leq n$, $|uv| \geq 1$, window anywhere.
- CFL closed: $\cup$, concat, star, $\cap$ regular. Not: $\cap$, complement.

COMPUTABILITY

- TM 7-tuple; halts when δ undefined.
- Regular $\subset$ DCFL $\subset$ CFL $\subset$ (CSL) $\subset$ Recursive $\subset$ RE.
- L and $\bar{L}$ RE $\Rightarrow$ L recursive. Halting: RE, not recursive; its complement not RE.
- Reduce known-undecidable $\rightarrow$ new problem.
- NP: poly verifier / NTM. NP-complete: in NP + reduction from known NPC.

HASKELL

- Signature, base case, recursive case, trace.
- Accumulator: `go xs acc`; `go [] acc = acc`.
- `foldl f acc (x:xs) = foldl f (f acc x) xs`
- `foldr f acc (x:xs) = f x (foldr f acc xs)`
- $x : xs$ is $O(1)$; $xs ++ ys$ is $O(|xs|)$.
- `listToInt`: `acc*10 + d`. `decompress`: `replicate n c ++ rest`.

PL CONCEPTS

- Encapsulation = grouping in one unit. Information hiding = internals inaccessible.
- Polymorphism: subtype, parametric, ad hoc.
- Lexer $\rightarrow$ tokens; parser $\rightarrow$ AST (+ precedence); executor $\rightarrow$ environments, call stack, heap.
- Value = copy; reference = alias; result; value-result; name.
- Static scope = text; dynamic = call chain.
- Semaphore wait/signal; monitor; messages; transactional memory.

Appendix B. Swedish–English glossary (PL exam is in Swedish)

Svenska	English	Svenska	English
uppgift	problem / question	poäng	points
konstruera	construct	kontextfri grammatik	context-free grammar
generera	generate	sträng / språk	string / language
icketerminal / terminal	non-terminal / terminal	startsymbol	start symbol
regel	rule / production	tomma strängen	the empty string
prefix	prefix	minst lika många	at least as many
hälften	half	dubbelt så många	twice as many
beskriv kortfattat	describe briefly	förklara	explain
precedens / associativitet	precedence / associativity	tolk / interpretator	interpreter
kompilator	compiler	datastruktur	data structure
tilldelning	assignment	binda ett värde till ett namn	bind a value to a name
sidoeffekt	side effect	skillnad	difference
för- och nackdelar	advantages and disadvantages	värdeanrop / referensanrop	call by value / by reference
inkapsling	encapsulation	informationsdöljande	information hiding
arv	inheritance	dynamisk bindning	dynamic binding
skriv en funktion	write a function	hjälpfunktion	helper function
inbyggda funktioner	built-in functions	tidskomplexitet, linjär	time complexity, linear
vänder på listor	reverses lists	svansrekursiv	tail-recursive
upprepningar	repetitions	returvärde	return value
jämnlöpande / samtidighet	concurrent / concurrency	avbrott / undantag	interrupt / exception
räckvidd	scope	livslängd	lifetime